BLDC MOTOR

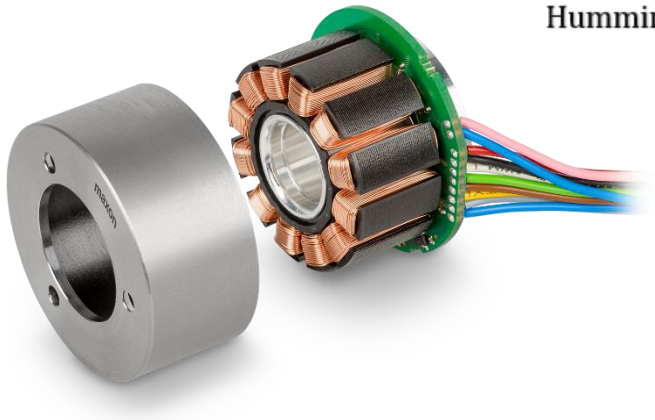
同济大学 – July 2025

Flavio Davide Borriello (2359270)

Francesco Tassi (2359266)

HBirdv2 E203

HummingBirdv2 E203 RISC-V Core and SoC



Intelligent Chip Final Exam Project
Automation Engineering bachelor's degree
College of Electronics and Information Engineering
Professor Jiang Lei

TABLE OF CONTENTS

THE ASSIGNMENT 1

Equipment used 2
Brushless motor & H-bridge controller 2

HDL IMPLEMENTATION OF THE BLDC CONTROLLER ... 3

Design Philosophy: A Hierarchical and Modular Approach 3
Register File (register_file.v) 5
ICB Slave Interface (icb_interface.v) 9
Sine Look-Up Table (sine_rom.v) 11
SPWM Phase Generator (spwm_phase_generator.v) 12
SPWM Engine (spwm_engine.v) 14
PWM Waveform Generator (pwm_waveform_generator.v) 16
Dead-Time Insertion for H-Bridge Protection 19
- *dead_zone_channel.v* 20
- *dead_zone_inserter.v* 20

Brake Logic (brake_logic.v) 24
HDL testbench: testing the hardware's code 28

CPU: HUMMINGBIRD E203 33

Internal bus 33
Connecting the BLDC Driver to the System Bus 34
GPIO assignments 36

THE C CODE 38

Bldc_driver.h 40
Main.c 41
Firmware Design and Verification 42
C Code Structure and Principles 43
Static and 6-Step PWM Mode Firmware 44
Sinusoidal PWM (SPWM) Mode Firmware 45

FINAL TESTING 46

Running the make command and testing 46
Static Mode Verification 49
PWM Mode Verification 50
Sinusoidal PWM (SPWM) Mode Verification 50

GOWIN IMPLEMENTATION 51

CONCLUSION 53

REFERENCES&SOURCES 54

THE ASSIGNMENT

The aim of this project is to design a controller module for a Brushless DC (BLDC) or Permanent Magnet Synchronous Motor (PMSM), integrated within a RISC-V-based system. Specifically, the controller is to be developed as a custom Intellectual Property (IP) core compatible with the ICB bus of the Hummingbird E203, a compact and efficient implementation of the RISC-V architecture. The controller is responsible for generating the three-phase Pulse Width Modulation (PWM) signals required to drive the motor. This involves designing a 16-bit timer equipped with six Output Compare (OC) registers, which produce six signals organized into three complementary pairs corresponding to the U, V, and W motor phases. Each pair consists of two signals that must be driven in a complementary fashion to operate the motor correctly. A critical feature of the design is the ability to configure a dead time, or "dead zone", between these complementary signals for each phase, ensuring safe switching and preventing shoot-through in the external power stage. Additionally, the controller must support the periodic updating of OC register values to generate a sinusoidal PWM pattern, essential for smooth and efficient motor operation. This updating can be handled either through software, using timer interrupts, or directly in hardware, depending on the chosen implementation approach. The system must also include an emergency stop mechanism—referred to as BREAK input—which immediately halts the motor when triggered. While the basic features of the controller can be verified through simulation, the project also includes a set of extended objectives intended for FPGA-based validation with an actual

BLDC motor and 12V three-phase driver circuitry. These optional features involve implementing hardware logic for automatic OC value injection from a precomputed sine wave table, adding a computation module for dynamic driving power adjustment, and integrating speed control capabilities by modulating the timer frequency or sine table feed rate. Together, these enhancements aim to create a versatile and robust motor control solution based on open-source hardware principles.

https://git.tongji.edu.cn/ichip/doc/-/blob/main/final_project/2025spring/prj3_bldc_ctrl.md

EQUIPMENT USED

The development and testing of the BLDC motor control system were carried out using a combination of hardware, software, and support tools. The primary processing platform was the **Tang Primer 20K FPGA board**, which hosted the custom digital logic for motor control. Motor actuation was achieved using a **NICHIBO MOTOR BLDC3640 12V M5 C-01679R021**, driven through a **6PWM BLDC motor power driver** capable of controlling all six transistors of a three-phase H-bridge independently. Firmware and HDL code were developed and edited using **Visual Studio Code**, while **Gowin Software** was employed for synthesis, implementation, and bitstream generation specific to the FPGA. Additionally, the team made use of the **Google Studio AI support tool** and **OpenAI's ChatGPT**, to assist with documentation drafting, technical language refinement, and clarification of relevant engineering concepts throughout the project.

BRUSHLESS MOTOR & H-BRIDGE CONTROLLER

A Brushless DC (BLDC) motor operates on the principle of electronically controlled commutation, eliminating the need for the mechanical brushes and commutator found in traditional DC motors. Its fundamental structure consists of a stationary set of windings, known as the stator, and a rotor embedded with permanent magnets.

The motor controller, which is the core of this project, sequentially energizes the stator windings to generate a rotating magnetic field. The permanent magnets on the rotor are attracted to this field and are continuously "pulled" along, creating torque and causing the rotor to spin in synchronization. To maintain continuous rotation, the controller must always know the precise position of the rotor to switch the current to the next set of windings correctly. In this implementation, rotor position is tracked using a high-resolution 16-bit counter, which provides precise timing information for commutation events without relying on physical sensors or back-electromotive force (back-EMF) measurements. There in fact are some sensors to detect the motor's position. They are called *hall sensors*, but have not been implemented on the project, that is however already compatible for this upgrade.

HDL IMPLEMENTATION OF THE BLDC CONTROLLER

DESIGN PHILOSOPHY: A HIERARCHICAL AND MODULAR APPROACH

To manage the complexity of the BLDC motor controller and ensure a robust, testable, and maintainable design, a hierarchical and modular approach was adopted for the Hardware Description Language (HDL) implementation. The controller is not a single, monolithic block of code; instead, it is broken down into many smaller, specialized sub-modules, each with a clearly defined purpose. This strategy allows for

"bottom-up" verification, where each individual unit can be tested in isolation before being integrated into the larger system.

The overall architecture of the BLDC controller IP consists of a top-level wrapper that instantiates and connects these core functional units. The key modules and their interconnections are as follows:

- **icb_interface.v:** This module acts as the translator between the E203 processor's complex Internal Chip Bus (ICB) protocol and the simple read/write signals required by our internal registers. It decodes the memory-mapped address from the CPU and manages the valid/ready handshake protocol.
- **register_file.v:** Serving as the nerve centre of the controller, this module holds all software-configurable parameters (such as operating mode, direction, period, and duty cycle) and hardware status information (like brake active status). It receives write commands from the icb_interface and provides its stored values to the other logic modules.
- **pwm_waveform_generator.v:** This is the primary logic engine for 6-step commutation. It contains the commutation state machine and the necessary comparators to generate the six "ideal" (pre-dead-time) PWM drive signals based on the configuration provided by the register_file.
- **spwm_engine.v:** A specialized module dedicated to Sinusoidal PWM (SPWM) mode. It contains the Sine Look-Up Table (LUT) and logic to generate three phase-shifted sinusoidal modulation values, which are then fed to the pwm_waveform_generator to create the final SPWM output.
- **dead_zone_inserter.v:** A critical safety module that takes the six ideal PWM signals from the pwm_waveform_generator. It guarantees hardware protection by inserting a small, programmable "dead-time" delay between the turn-off of a High-

Side switch and the turn-on of its complementary Low-Side switch, preventing H-bridge shoot-through.

- **brake_logic.v:** This safety-override module sits at the end of the signal chain. It takes the final, dead-time-corrected PWM signals and, if a hardware or software brake is asserted, it overrides them with a predefined braking pattern (either coasting or dynamic braking).
- **bldc_controller.v** top-level module encapsulates this entire system, instantiating each of these sub-modules and managing the data, control, and status signal paths between them, ultimately presenting a single, cohesive peripheral to the CPU and the external FPGA pins.

There are in addition two clock modules, **clock_prescaler.v** and **main_timer.v**, which synchronise the operations of the system. Particularly, the main_timer.v contains the 16-bit counter necessary for the PWM commutation.

We will now introduce each module in detail.

REGISTER FILE (REGISTER_FILE.V)

The register_file.v module serves as the central nervous system for the entire BLDC controller, acting as the direct hardware interface for the software running on the E203 processor. It is designed around a memory-mapped architecture where specific addresses, provided by the icb_interface, correspond to dedicated internal registers.

Based on the controller's requirements, a total of **six 32-bit registers** were implemented, each aligned to a 4-byte boundary. Notice how many areas of these registers are **reserved**, this means that those bits don’t carry any relevant information and are free for future applications. Five of the registers are read/write (R/W) configuration registers, while the sixth is a read-only (RO) status register:

- **0x00: CONTROL_REG (R/W):** Holds the primary operational commands, such as motor enable, direction, operating mode selection (Static, PWM, SPWM), and brake enable and mode and even soft reset.
- **0x04: CARRIER_TIMING_REG (R/W):** Contains the PWM period and the dead-zone duration value.
- **0x08: COMMUTATION_TIMING_REG (R/W):** Defines the duration of each of the six commutation steps when operating in non-sinusoidal modes.
- **0x0C: PWM_DUTY_REG (R/W):** Sets the generic PWM duty cycle value applied to the active motor phase.
- **0x10: SPWM_CONFIG_REG (R/W):** Contains parameters specific to Sinusoidal PWM mode (frequency and amplitude)
- **0x14: STATUS_REG (RO):** Allows the software to read back the live status of the controller, including whether the brake is active, the current commutation step, and other real-time hardware states.

A table to further illustrate the structure is in the following page.

BLDC Motor Register Map

Register	Field Name	Notes:
CONTROL_REG (0x00, R/W)	Op_Mode [1:0] Direction [2] Enable_Motor [3] Soft_Reset_Internal [4] SW_Brake_Enable [5] SW_Brake_Pattern [7:6] HW_Brake_Pin_Enable [8] Reserved [31:9]	-Software reset (write-1-to-set pulse) - Enable brake pin from hardware
CARRIER_TIMING_REG (0x04, R/W)	PWM_Carrier_Period [15:0] Dead_Zone_Value [19:16] Reserved [31:20]	Period of the PWM carrier timer and dead time insertion
COMMUTATION_TIMING_REG (0x08, R/W)	Commutation_Step_Period [15:0] Reserved [31:16]	16-bit counter
PWM_DUTY_REG (0x0C, R/W)	PWM_Duty_Value_Generic [15:0] Reserved [31:16]	PWM duty value for active phase
SPWM_CONFIG_REG (0x10, R/W)	SPWM_Phase_Increment [15:0] Reserved [23:16] SPWM_Amplitude_Global [31:24]	-phase inc. determines frequency -Amp. Determines amplitude
STATUS_REG (0x14, Read-Only)	Brake_Active_HwStatus [0] Motor_Enabled_HwStatus [1] Reserved [4:2] Current_Actual_Op_Mode_HwStatus [6:5] Current_Commutation_State_6Step_HwStatus [9:7] Reserved [15:10] Current_SPWM_Phase_Accumulator_U_HwStatus [31:16]	A lot of data to check the motor's functioning

The Verilog implementation uses a synchronous always block for handling register writes, ensuring that all configuration registers are updated predictably on the rising edge of the clock when a valid write enable (reg_write_en) is present. For read operations, the module employs a combinational always @(*) block, as shown in the code snippet. This block acts as a multiplexer, selecting which register's data to output onto reg_read_data_muxed based on the reg_addr_sel input. A key feature of this implementation is how the STATUS_REG is handled. Unlike the other registers that store static values, the STATUS_REG is constructed "on-the-fly" within this combinational block. When the CPU requests to read the status address, the logic gathers the live hardware status signals from its various inputs (e.g., brake_active_hw_in, current_comm_state_hw_in) and assembles them into a 32-bit word. This design ensures that the CPU always receives the most up-to-date, real-time status of the controller without introducing any latency from register clocking.

register_file.v

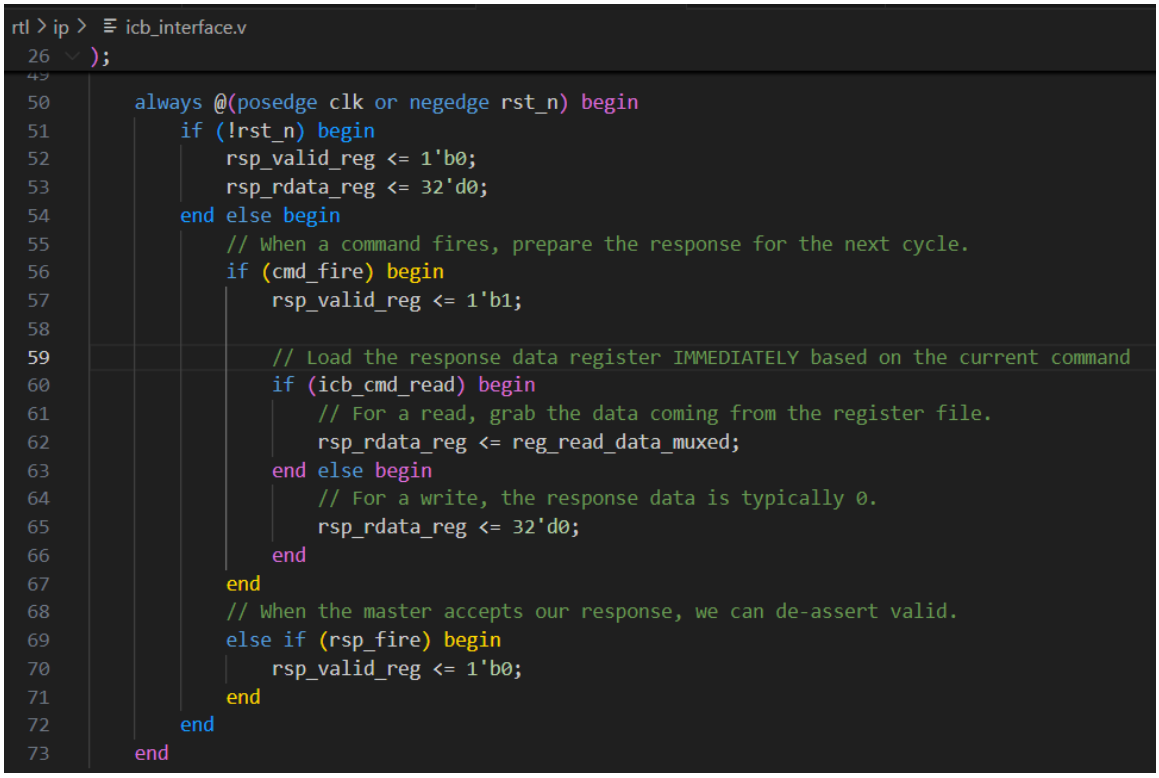
```
46  );
127
128     // --- Register Read Logic (Combinational) ---
129     always @(*) begin
130         // The case statement must assign to reg_read_data_muxed in all branches
131         // to avoid inferring a latch. The default case handles unmapped addresses.
132         case (reg_addr_sel)
133             ADDR_CONTROL:             reg_read_data_muxed = REG_CONTROL;
134             ADDR_CARRIER_TIMING:      reg_read_data_muxed = REG_CARRIER_TIMING;
135             ADDR_COMMUTATION_TIMING:  reg_read_data_muxed = REG_COMMUTATION_TIMING;
136             ADDR_PWM_DUTY:            reg_read_data_muxed = REG_PWM_DUTY;
137             ADDR_SPWM_CONFIG:         reg_read_data_muxed = REG_SPWM_CONFIG;
138             ADDR_STATUS: begin
139                 // Construct the Status Register value directly into reg_read_data_muxed
140                 // Initialize all bits first to ensure no latches for unassigned bits within this scope
141                 reg_read_data_muxed = 32'b0; // Default to all zeros for this specific case branch
142
143                 reg_read_data_muxed[0]     = brake_active_hw_in;
144                 reg_read_data_muxed[1]     = motor_enabled_actual_hw_in;
145                 // Reserved [4:2] are already 0 due to the line above
146                 reg_read_data_muxed[6:5]   = current_op_mode_hw_in;
147                 reg_read_data_muxed[9:7]   = current_comm_state_hw_in;
148                 // Reserved [15:10] are already 0
149                 reg_read_data_muxed[31:16] = current_spwm_phase_u_hw_in;
150             end
151             default:                  reg_read_data_muxed = 32'hDEADBEEF;
152         endcase
153     end
```

ICB SLAVE INTERFACE (ICB_INTERFACE.V)

For the BLDC motor controller to be a useful peripheral within the System-on-Chip, it must be able to receive commands and report status back to the central E203 processor. The icb_interface.v module serves as this essential communication bridge. It functions as a "slave" device on the processor's Internal Chip Bus (ICB), translating the bus protocol's timed transactions into simple, single-cycle read and write signals that our internal register_file can easily understand.

The primary responsibilities of this module are to manage the valid/ready handshake protocol for both commands and responses. When the E203 (the bus master) asserts icb_cmd_valid and places an address on the bus, our interface first performs address decoding. It compares the upper bits of the incoming address with its own hardcoded BASE_ADDR to determine if the command is intended for this peripheral (an "address hit"). If the address matches, the interface asserts icb_cmd_ready, signaling to the master that it is ready to accept the transaction. A command is only "fired" and accepted on a clock edge when both valid and ready are high, ensuring a synchronized data transfer. The interface then generates the necessary internal signals—a reg_write_en pulse for write operations or a reg_addr_sel for read operations—to interact with the register_file.

This module employs a simple yet powerful pipelined state machine to manage the response channel and ensure correct timing. The core of this logic, shown in the code snippet, is managed by the rsp_valid_reg register.



When a command "fires" (cmd_fire is high for one clock cycle), two things happen simultaneously. First, rsp_valid_reg is scheduled to be set to 1 on the next clock edge. This creates a **one-cycle response latency**, which is a standard feature of many bus protocols. The peripheral takes one cycle to process the request and prepare the response. Second, and crucially, the response data register (rsp_rdata_reg) is immediately loaded with the appropriate value. If the command was a read (icb_cmd_read is high), it captures the data currently being output by the register_file's multiplexer (reg_read_data_muxed). If it was a write, it is simply loaded with zero. On the subsequent clock cycle, icb_rsp_valid (which is assigned from rsp_valid_reg) goes high, presenting both the valid signal and the correct, stable response data to the master simultaneously.
The rsp_valid_reg remains high, indicating a pending response, until the master signals its acceptance by
asserting icb_rsp_ready (making rsp_fire high), at which

point rsp_valid_reg is de-asserted, completing the handshake and making the peripheral ready for a new command.

SINE LOOK-UP TABLE (SINE_ROM.V)

At the very core of the Sinusoidal Pulse Width Modulation (SPWM) engine lies the sine_rom.v module, which functions as a simple, read-only memory. This memory is the foundational data source, storing a pre-calculated, digitized representation of a sine wave's shape. The ROM is implemented as a 64-entry array, with each entry holding an 8-bit unsigned integer. These values represent the amplitude of a sine wave at discrete points in time. By reading through these values sequentially, the controller can reconstruct a digital version of the sinusoidal waveform that is required to drive the motor smoothly.

A key design choice for this module is the use of a **quarter-wave Look-Up Table (LUT)**. Instead of storing the values for a full 360-degree sine wave, the ROM only contains the 64 data points corresponding to the first 90 degrees (one-quarter) of the wave. This approach significantly reduces the memory resources required on the FPGA. The full 360-degree waveform is then ingeniously reconstructed by the parent module (spwm_phase_generator.v) by leveraging the inherent symmetry of the sine function. The second quadrant (90-180°) is a mirror image of the first, and the third and fourth quadrants (180-360°) are simply the inverted (negative) versions of the first two. By applying simple logic to read the table forwards or backwards and selectively negating the output, the full, signed sine wave can be generated from this compact, resource-efficient LUT.

The sine_rom module itself is purely passive. It is instantiated by its parent module, spwm_phase_generator, which is responsible for all control and addressing. The parent module calculates the appropriate 6-bit memory address based on the desired point in the waveform and provides this addr to the ROM. In response,

the sine_rom combinationally outputs the corresponding 8-bit data value (q) from its internal memory array. This simple, direct read mechanism ensures that the sine wave data is available instantly to the parent module for further processing. During FPGA synthesis, this Verilog description of a memory is typically implemented using the dedicated, high-speed Block RAM (BRAM) resources available on the chip for maximum performance.

```
6  module sine_rom (
7      input      [5:0]  addr,    // Address (0-63)
8      output reg [7:0]  q        // Data Out (8-bit sine value)
9  );
10
11     // Memory declaration
12     reg [7:0] mem [0:63];
13
14     // Read logic
15     always @(*) begin
16         q = mem[addr];
17     end
18     // Memory initialization
19     initial begin
20 // Corrected full-range values (0-255) for the sine LUT
21 mem[0] = 8'd0;
22 mem[1] = 8'd6;
23 mem[2] = 8'd13;
24 mem[3] = 8'd19;
25 mem[4] = 8'd25;
```

SPWM PHASE GENERATOR (SPWM_PHASE_GENERATOR.V)

The spwm_phase_generator module serves as the primary engine for creating the fundamental three-phase signed sine wave outputs (sine_out_u, sine_out_v, sine_out_w), each phase-shifted by 120 degrees from the others. This is achieved through three key internal processes: phase accumulation for frequency control, instantiation of the previously described Sine ROM for wave shaping, and quarter-

wave reconstruction logic to build the full 360-degree sine wave from the values for only 90-degree of it.

The module's operation begins with a high-resolution phase accumulator, a 32-bit register that is incremented by the frequency_step_in value on every clock cycle. The rate at which this accumulator overflows determines the fundamental frequency of the output sine waves; a larger frequency step results in a faster overflow and thus a higher output frequency. The top 8 bits of this accumulator are used to represent the current electrical angle for Phase U. The angles for Phase V and Phase W are then simply derived by adding constant 120-degree and 240-degree offsets to the Phase U angle, ensuring the correct three-phase relationship.

The actual sine wave shape is generated using a highly efficient technique known as quarter-wave synthesis. Instead of storing a full 360-degree sine wave in memory, we only store the first quadrant (0 to 90 degrees) in a small sine_rom module.
The spwm_phase_generator then cleverly reconstructs the full sine wave for each phase using the logic shown in the code snippet. This logic uses the top two bits of the current angle to determine which of the four quadrants the wave is in. Based on the quadrant, it either reads the ROM forwards or backwards (by inverting the address bits to create a mirror image) and either keeps the output positive or negates it. This method drastically reduces the required memory resources while perfectly recreating a symmetrical, signed, full-period sine wave. The logic for each of the three phases is identical, simply operating on its own 120-degree-shifted angle, which guarantees the generation of three perfectly phase-aligned sinusoidal outputs. Here is a demonstration of such logic:

spwm_phase_generator.v

```
70   always @(*) begin
80       //------------------------------------------------
81       // Phase U Logic
82       //------------------------------------------------
83       // Use the top 2 bits of the angle to select the quadrant
84       case (angle_u[7:6])
85           // Quadrant 1 (0-89 deg): Read ROM forward, output is positive
86           2'b00: begin
87               rom_addr_u = angle_u[5:0];
88               sine_out_u = rom_q_u; // Implicitly cast unsigned 8-bit to signed 9-bit
89           end
90           // Quadrant 2 (90-179 deg): Read ROM backward, output is positive
91           2'b01: begin
92               rom_addr_u = ~angle_u[5:0]; // Invert bits for mirror image
93               sine_out_u = rom_q_u;
94           end
95           // Quadrant 3 (180-269 deg): Read ROM forward, output is negative
96           2'b10: begin
97               rom_addr_u = angle_u[5:0];
98               sine_out_u = -rom_q_u; // Negate the value
99           end
100          // Quadrant 4 (270-359 deg): Read ROM backward, output is negative
101          2'b11: begin
102              rom_addr_u = ~angle_u[5:0];
103              sine_out_u = -rom_q_u;
104          end
105          default: begin // Should not happen
106              rom_addr_u = 6'd0;
107              sine_out_u = 9'd0;
108          end
109      endcase
```

SPWM ENGINE (SPWM_ENGINE.V)

The spwm_engine.v module serves as the high-level management and calculation engine for the entire Sinusoidal PWM (SPWM) generation process. Its primary function is to act as a bridge between the software-defined control parameters and the low-level hardware that generates the waveforms. It takes a single 32-bit configuration register containing both frequency and amplitude information, along with the PWM carrier period, and outputs three final, unsigned "Output Compare" (OC) values. These OC values directly represent the required duty cycle for each of the three motor phases at any given moment.

Upon receiving the 32-bit spwm_config_in from the system's control registers, the engine first deconstructs this word into its two constituent parts: a 16-bit frequency_step and an 8-bit amplitude. It then passes the frequency_step to its child module, the spwm_phase_generator, which returns three pure, signed sine waves, each correctly phase-shifted by 120 degrees.

The final and most critical stage within the spwm_engine is the mathematical transformation of these signed sine waves into practical, unsigned duty cycle values. Each signed sine wave is first multiplied by the amplitude value, allowing software to control the overall voltage (and thus torque) applied to the motor. To convert the resulting bipolar (positive and negative) scaled wave into a unipolar signal suitable for a PWM duty cycle comparator, a DC offset is added. This offset is calculated as half of the PWM carrier period (pwm_period_in >> 1), effectively shifting the entire sine wave into a positive range where it can be directly compared against the PWM carrier timer. The result is three clean, correctly scaled, and biased duty cycle values ready for the final PWM generation stage.

```
67   // Perform the amplitude scaling via multiplication
68   // Use $signed() to ensure the multiplication is a signed operation.
69   assign scaled_sine_u = sine_out_u_signed * $signed(amplitude);
70   assign scaled_sine_v = sine_out_v_signed * $signed(amplitude);
71   assign scaled_sine_w = sine_out_w_signed * $signed(amplitude);
72
73   // Normalize the scaled value by shifting right by 8 bits
74   // This is equivalent to dividing by 256.
75   assign normalized_sine_u = scaled_sine_u >>> 8; // Use arithmetic shift for signed numbers
76   assign normalized_sine_v = scaled_sine_v >>> 8;
77   assign normalized_sine_w = scaled_sine_w >>> 8;
78
79   // Add the DC offset to level-shift the wave into an unsigned range
80   // The result is the final duty cycle value.
81   assign oc_value_u = $signed(dc_offset) + normalized_sine_u;
82   assign oc_value_v = $signed(dc_offset) + normalized_sine_v;
83   assign oc_value_w = $signed(dc_offset) + normalized_sine_w;
84
85
86   endmodule
```

It's time to connect to pwm_waveform_generator.v, the grand master!

PWM WAVEFORM GENERATOR (PWM_WAVEFORM_GENERATOR.V)

To operate a Brushless DC (BLDC) motor, it is necessary to apply electrical energy to its three stator phases in a precise and time-dependent sequence. This is typically achieved through a process called **commutation**, which involves switching current among the motor's windings to generate a rotating magnetic field that drives the rotor. The technique is called "**Pulse Width Modulation**" (PWM) and it controls the average voltage delivered to the motor by rapidly switching power ON and OFF. The key parameter is the **duty cycle**—the percentage of time the power is ON within a high-frequency cycle. A higher duty cycle results in a higher average voltage, increasing speed and torque. Our controller implements three distinct, software-selectable modes that leverage this principle in progressively sophisticated ways.

1. **Static Mode:** This is the most basic form of 6-step commutation. For each of the six steps, the active High-Side switch is held ON continuously (a 100% duty cycle), delivering full voltage for basic, full-torque rotation.

2. **PWM Mode:** This mode enhances 6-step commutation by applying a PWM signal with a fixed but variable duty cycle to the active High-Side switch. This allows for fine-grained control over the average voltage, enabling smooth speed and torque adjustment.

3. **Sinusoidal PWM (SPWM) Mode:** This is the most advanced method. Instead of a fixed duty cycle per step, the duty cycle is continuously modulated to follow the amplitude of a sine wave. Three such PWM signals, each phase-shifted by 120°, generate a smoothly rotating magnetic field. This results in significantly smoother torque, lower noise, and higher efficiency, which is the primary goal of this project.

All three strategies are managed by a central hardware module that allows the software to toggle between modes as needed.

Pwm_waveform_generator.v

The pwm_waveform_generator module represents the core logic responsible for generating the ideal High-Side (HS) drive signals for all three motor phases across the various supported modes of operation. It acts as a configurable state machine and signal multiplexer, taking high-level control inputs—such as the desired operating mode, direction, and period/duty cycle values—and translating them into the precise PWM waveforms required. This module intelligently manages different timing and signal generation strategies by instantiating and controlling specialized sub-modules for both 6-step and Sinusoidal PWM (SPWM) operation.

Operating Modes and Navigation

The generator is designed to operate in three distinct software-selectable modes, which are chosen via the op_mode_in input from the system's control registers. The module uses a primary case statement to navigate between these modes, ensuring that only the logic relevant to the currently selected mode is used to drive the final outputs.

1. **MODE_STATIC (6-Step, Full Voltage):** In this fundamental mode, the module uses a simple commutation state machine driven by a step_timer. The timer dictates the duration of each of the six commutation steps. For each step, the logic asserts the High-Side output of the corresponding active phase to a

constant high (1'b1), effectively delivering 100% of the available voltage.

```
rtl > ip > pwm_waveform_generator.v
 33 );
 93     always @(*) begin
 99         if (enable_motor_in) begin
100             case (op_mode_in)
101                 MODE_STATIC: begin
102                     case (commutation_state)
103                         3'd0: ideal_hs_u_out = 1'b1; // U+ (V- implied to be 0)
104                         3'd1: ideal_hs_u_out = 1'b1; // U+ (W- implied to be 0)
105                         3'd2: ideal_hs_v_out = 1'b1; // V+ (W- implied to be 0)
106                         3'd3: ideal_hs_v_out = 1'b1; // V+ (U- implied to be 0)
107                         3'd4: ideal_hs_w_out = 1'b1; // W+ (U- implied to be 0)
108                         3'd5: ideal_hs_w_out = 1'b1; // W+ (V- implied to be 0)
109                         default: ; // All ideal_hs outputs remain 0
110                     endcase
111                 end
```

2. **MODE_PWM (6-Step, Variable Voltage):** This mode builds directly upon the static 6-step commutation sequence. While the step_timer still determines the sequence and duration of which phase is active, the output is no longer a constant high. Instead, the High-Side signal for the active phase is generated by comparing the count of a high-frequency pwm_carrier_timer against a software-defined generic_pwm_duty_val_in.

```
112                 MODE_PWM: begin
113                     // Apply the generic_pwm_duty_val_in to the currently active HS phase
114                     // The other two phases' ideal_hs outputs remain 0 (defaulted above)
115                     case (commutation_state)
116                         3'd0: ideal_hs_u_out = (internal_pwm_timer_count < generic_pwm_duty_val_in);
117                         3'd1: ideal_hs_u_out = (internal_pwm_timer_count < generic_pwm_duty_val_in);
118                         3'd2: ideal_hs_v_out = (internal_pwm_timer_count < generic_pwm_duty_val_in);
119                         3'd3: ideal_hs_v_out = (internal_pwm_timer_count < generic_pwm_duty_val_in);
120                         3'd4: ideal_hs_w_out = (internal_pwm_timer_count < generic_pwm_duty_val_in);
121                         3'd5: ideal_hs_w_out = (internal_pwm_timer_count < generic_pwm_duty_val_in);
122                         default: ; // All ideal_hs outputs remain 0
123                     endcase
124                 end
```

3. **MODE_SPWM (Sinusoidal PWM):** In this most advanced mode, the module delegates waveform generation to a dedicated spwm_engine instance. The 6-step commutation logic is bypassed entirely. The spwm_engine module generates three phase-shifted sinusoidal duty cycle values. It is important to

initiate such module within the pwm_generator.v file. The spwm modules have been described in other paragraphs.

By encapsulating these three distinct operational strategies within a single module, the pwm_waveform_generator provides a flexible and powerful interface for the central processor to execute a wide range of motor control algorithms.

The waveforms of the three modes are shown in the final testing at the end of this report.

DEAD-TIME INSERTION FOR H-BRIDGE PROTECTION

In any H-bridge or half-bridge topology used for motor control, each phase of the motor is driven by a pair of complementary switches—a High-Side (HS) switch connecting the phase to the positive power rail and a Low-Side (LS) switch connecting it to ground. In an ideal scenario, one switch would turn off at the exact same instant its complement turns on. However, physical transistors do not switch instantaneously; they have finite turn-on and turn-off times.

If a command is given to turn the LS switch on before the HS switch has fully turned off (or vice-versa), there will be a *brief but catastrophic* moment where both switches are simultaneously conducting. This creates a direct, low-impedance path from the power supply to ground through the two switches, a condition known as shoot-through. Shoot-through results in a massive current spike that can permanently damage the power switches, the driver IC, and even the power supply itself.

To prevent this destructive phenomenon, it is absolutely essential to implement **dead-time**. Dead-time is a short, deliberately inserted delay between the command to turn one switch OFF and the subsequent command to turn its complementary switch ON. During this interval, both the High-Side and Low-Side switches for that phase are guaranteed to be off, ensuring the "break-before-make" operation critical for hardware safety. The duration of this dead-time must be

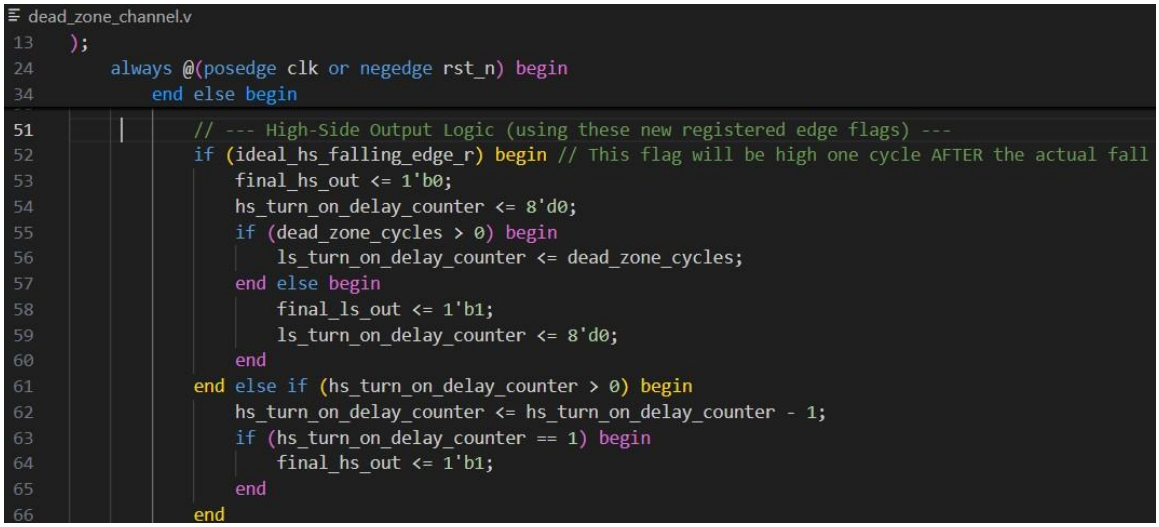
carefully chosen to be longer than the turn-off time of the transistors but as short as possible to minimize waveform distortion. For our design, this critical safety feature is software-programmable, allowing for fine-tuning based on the specific H-bridge characteristics.

To implement the dead-time logic in a clean, modular, and reusable manner, we adopted a hierarchical two-module design pattern. This approach separates the core single-phase logic from the top-level three-phase management, making the design easier to develop, test, and understand.

- ***dead_zone_channel.v*** (The "Worker" Module): This module constitutes the core engine for dead-time insertion for a single motor phase. It is designed to take one "ideal" High-Side PWM signal as its primary input. Internally, it detects the rising and falling edges of this ideal signal. Upon detecting an edge, it immediately turns the corresponding switch OFF while simultaneously starting a programmable-delay counter for the complementary switch. Only after this dead-time delay has elapsed is the complementary switch commanded to turn ON. This module is the fundamental building block responsible for generating two safe, non-overlapping final output signals (HS and LS) for one phase.
- ***dead_zone_inserter.v*** (The "Manager" Module): This module serves as a high-level structural wrapper that manages the dead-time for the entire three-phase system. Its primary role is to provide a clean interface for the motor controller. Internally, its logic is very simple: it instantiates three separate copies of the dead_zone_channel.v worker module—one for phase U, one for phase V, and one for phase W. It takes the three ideal High-Side signals from the main PWM generator and the global dead_zone_value from the control registers and correctly routes them to the respective child instances. This design ensures that each phase is handled independently and encapsulates the

complexity, presenting a simple "dead-time insertion" block to the top-level design.

This two-tiered strategy allowed us to first rigorously test the complex timing logic of a single channel in isolation (unit testing) before integrating it into the full three-phase system, leading to a more robust and verifiable design.



```
dead_zone_channel.v
13   );
24       always @(posedge clk or negedge rst_n) begin
34           end else begin
51               // --- High-Side Output Logic (using these new registered edge flags) ---
52               if (ideal_hs_falling_edge_r) begin // This flag will be high one cycle AFTER the actual fall
53                   final_hs_out <= 1'b0;
54                   hs_turn_on_delay_counter <= 8'd0;
55                   if (dead_zone_cycles > 0) begin
56                       ls_turn_on_delay_counter <= dead_zone_cycles;
57                   end else begin
58                       final_ls_out <= 1'b1;
59                       ls_turn_on_delay_counter <= 8'd0;
60                   end
61               end else if (hs_turn_on_delay_counter > 0) begin
62                   hs_turn_on_delay_counter <= hs_turn_on_delay_counter - 1;
63                   if (hs_turn_on_delay_counter == 1) begin
64                       final_hs_out <= 1'b1;
65                   end
66               end
```

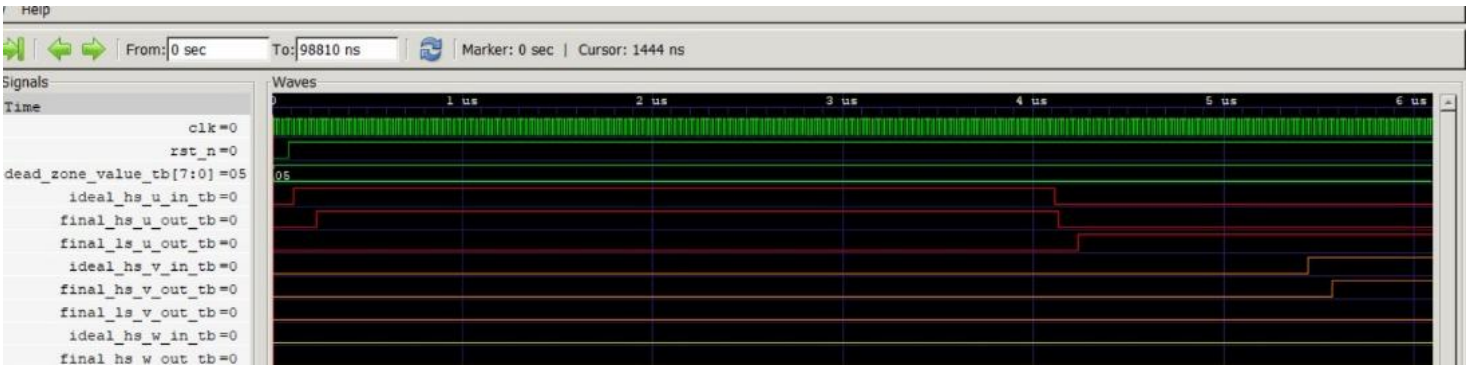
The core dead-time mechanism is implemented within a synchronous always block, as shown in the code snippet for the High-Side output logic. This logic is triggered by registered edge-detection flags to ensure safe, predictable state transitions. When the ideal_hs_falling_edge_r flag is asserted (indicating the ideal High-Side signal has just transitioned from high to low), the module performs two critical actions. First, it immediately forces the final High-Side output (final_hs_out) to a low state, initiating the "break" portion of the "break-before-make" sequence. Second, it prepares the complementary Low-Side switch for activation by loading its delay counter (ls_turn_on_delay_counter) with the software-programmed dead_zone_cycles value. Symmetrical but opposite logic exists for handling a rising edge, which turns off the Low-Side switch and loads the High-Side's delay counter.

The subsequent else if condition manages the turn-on delay itself. This block is active only when a turn-on sequence has been initiated and the corresponding delay counter is non-zero. On each clock cycle, it decrements the counter. Only when the counter reaches its final tick (== 1) is the final_hs_out signal asserted high on the next clock edge. This countdown process creates the essential time gap where both switches are off. This event-driven, counter-based approach guarantees that a complementary switch is never commanded to turn on until after its counterpart has been turned off and the specified dead-time has fully elapsed, robustly preventing any possibility of H-bridge shoot-through.

Passing on to the dead_zone_inserter.v module, it serves as a high-level, structural wrapper whose sole purpose is to manage the dead-time for the entire three-phase system in a clean and organized manner. It does not contain any complex timing logic itself; instead, it accomplishes its task by instantiating the dead_zone_channel.v "worker" module three separate times—one for each motor phase: U, V, and W.

This parent module provides a simple, unified interface to the rest of the controller. It takes the three "ideal" High-Side signals (ideal_hs_u_in, ideal_hs_v_in, ideal_hs_w_in) and the single, global dead_zone_value_in as inputs. Internally, it passes the clock, reset, and the global dead-zone value down to all three child instances. Crucially, it routes each phase's specific ideal signal to its corresponding dead_zone_channel instance. The final, shoot-through-protected High-Side and Low-Side outputs from each child instance are then wired directly to the top-level output ports of this module. This design elegantly encapsulates the complexity of the three-phase system, presenting a single, easy-to-use "dead-time insertion" block to the main controller.

Let’s now admire the results of such modules:



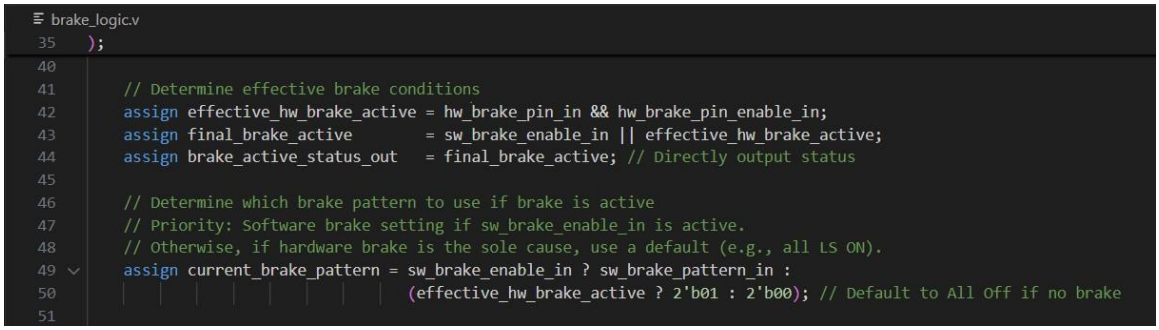
The provided waveform clearly demonstrates the successful operation of the dead-time insertion logic on the U-phase signals. The ideal_hs_u_out_tb (top red signal) represents the ideal command from the PWM engine. At approximately 120 nanoseconds, this ideal signal transitions from low to high, requesting the High-Side switch to turn on. The final_hs_u_out_tb (middle red signal) does not follow immediately. Instead, the dead_zone_channel module enforces a delay, keeping the output low. After the programmed dead-time of 100 ns (5 cycles of the 20 ns clock) has elapsed, the final_hs_u_out_tb signal safely transitions to high at approximately 220 ns. During this entire turn-on sequence, the final_ls_u_out_tb (bottom red signal) correctly remains low, preventing any possibility of shoot-through.

The symmetrical "break-before-make" behavior is showcased at the falling edge of the ideal signal, around the 4.1-microsecond mark. Here, ideal_hs_u_out_tb transitions from high to low. The final_hs_u_out_tb output responds instantly, going low on the next clock cycle to immediately turn off the High-Side switch. This action initiates the dead-time period for the complementary Low-Side switch. The waveform shows that for the next 100 ns, both the final_hs_u_out_tb and final_ls_u_out_tb signals are held low. Only after this critical safety interval has passed does the final_ls_u_out_tb signal transition to high, safely turning on the Low-Side switch. This

precisely controlled, non-overlapping behavior is fundamental to protecting the H-bridge hardware. Of course, the same is true also for V and W signals, which are not visible in this small screenshot of the simulation, which is pretty long and only tests a single wave at a time.

BRAKE LOGIC (BRAKE_LOGIC.V)

This section of the brake_logic module contains the core combinatorial decision-making logic, which is responsible for determining the overall braking status and the specific braking pattern to be applied. The logic is implemented using continuous assign statements to ensure an immediate response to any change in the control inputs. First, the effective_hw_brake_active signal is determined by a logical AND between the physical hardware brake pin (hw_brake_pin_in) and its corresponding enable flag, ensuring the hardware pin is only considered when it is configured to be active. The final, overarching brake status (final_brake_active) is then established by a logical OR of the software-commanded brake (sw_brake_enable_in) and this effective hardware brake status. This final_brake_active signal is also directly assigned to the brake_active_status_out port, providing real-time status feedback to the system. Subsequently, the current_brake_pattern is selected using a ternary operator (? :) that elegantly implements a priority system: if the software brake is enabled, the pattern specified by sw_brake_pattern_in takes precedence. If not, and the brake is triggered solely by the hardware pin, a default pattern (2'b01 for dynamic braking) is used, guaranteeing a safe and predictable emergency stop behavior. This hierarchical and combinatorial approach ensures that braking decisions are made robustly and without latency.



This brake_logic.v module is a crucial component for ensuring safe and controlled deceleration or stoppage of the BLDC motor. It acts as an override mechanism, taking precedence over the normal Pulse Width Modulation (PWM) signals generated by the main motor control logic (which are assumed to have already passed through dead-time insertion). The module provides flexibility by allowing braking to be initiated either by a software command from the E203 processor or by an external hardware signal, such as an emergency stop button on the Tang Primer 20K board. This dual-trigger capability ensures both programmed and immediate physical intervention can halt the motor.

The module intelligently determines if a brake condition is active by checking both a software-enable flag (sw_brake_enable_in) and the state of a hardware brake pin (hw_brake_pin_in, if enabled by hw_brake_pin_enable_in). If either of these sources indicates a brake request, the final_brake_active signal is asserted. When a brake is active, the module then refers to the current_brake_pattern. This pattern is primarily determined by a software-configurable setting (sw_brake_pattern_in) if the software brake is enabled, giving the CPU fine-grained control over the braking method. However, if the brake is triggered solely by the hardware pin, the module defaults to a predefined safe and effective braking pattern (all Low-Side switches ON). This hierarchical approach ensures that a deliberate software choice for braking is respected, while still providing a robust default for hardware-initiated emergency stops. The brake_active_status_out

signal provides feedback to the system, indicating whether the motor is currently under a braking condition.

When braking is engaged, the module's core logic, implemented as a combinational always block with a case statement, selects one of the predefined braking strategies. The primary strategies implemented are:

1. All Off (Coasting): All six final PWM outputs (High-Side and Low-Side for all three phases) are driven low. This effectively disconnects the motor windings from the power supply and ground, allowing the motor to freewheel or coast to a stop, driven only by its inertia and mechanical friction. This is a gentle way to stop and consumes minimal power during the braking phase itself.

2. All High-Sides Off / All Low-Sides On (Dynamic Braking): In this mode, all High-Side outputs are driven low, and all Low-Side outputs are driven high. This creates a short circuit across the motor windings through the Low-Side MOSFETs. As the motor's rotor continues to spin due to inertia, it acts as a generator, inducing a back-EMF. This voltage drives current through the shorted windings and the Low-Side switches, dissipating energy (typically as heat in the motor windings and switches) and creating a counter-electromagnetic torque that actively brakes the motor. This method provides a more aggressive and faster stop compared to coasting.

Reserved or undefined brake patterns default to the "All Off" state for safety. If no brake condition is active, the module simply passes the incoming PWM signals (pwm_hs_u_in, pwm_ls_u_in, etc.) directly to its outputs (final_pwm_hs_u_out, final_pwm_ls_u_out, etc.), allowing normal motor operation.

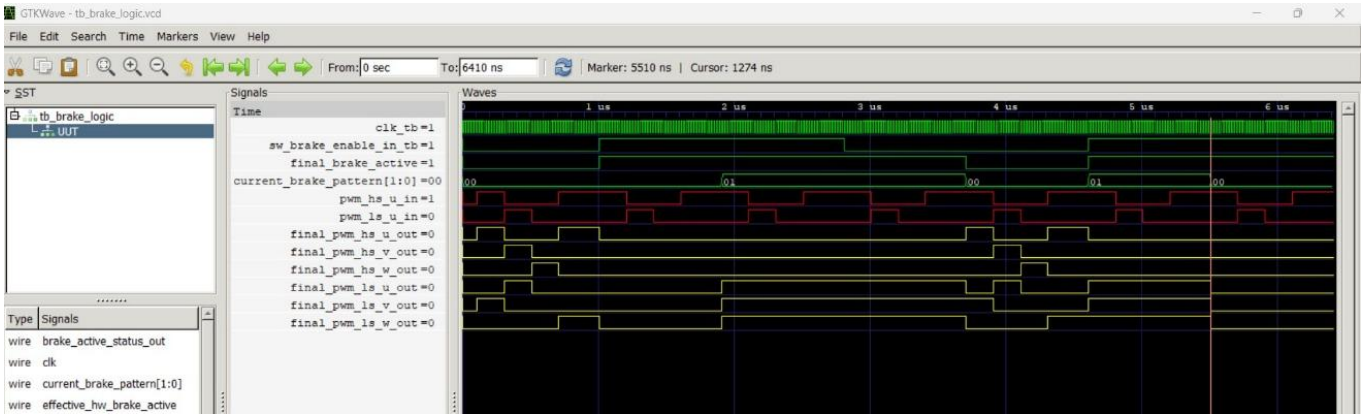
Because we have a lot of register space, we chose to keep two bits for breaking mode in order to have room for two more breaking mode implementations in the future.

The decision to implement braking in this manner, as a distinct module that overrides the final drive signals, offers several advantages. It provides a clear and centralized point for all braking logic, making it easier to manage and verify. By operating on signals that are presumed to be post-dead-time insertion, it simplifies the brake module's internal logic as it doesn't need to re-implement dead-time for its specific brake patterns (which are generally static states like all LS ON). The separation of software and hardware triggers, along with selectable braking patterns, provides a robust and versatile braking system suitable for various operational and safety requirements in a BLDC motor control application.

brake_logic.v

```
35    );
52        // Output logic (Combinational)
53        always @(*) begin
54            if (final_brake_active) begin
55                case (current_brake_pattern)
56                    2'b00: begin // All Off (Coast)
57                        final_pwm_hs_u_out = 1'b0; final_pwm_ls_u_out = 1'b0;
58                        final_pwm_hs_v_out = 1'b0; final_pwm_ls_v_out = 1'b0;
59                        final_pwm_hs_w_out = 1'b0; final_pwm_ls_w_out = 1'b0;
60                    end
61                    2'b01: begin // All HS Off / All LS On (Dynamic Brake)
62                        final_pwm_hs_u_out = 1'b0; final_pwm_ls_u_out = 1'b1;
63                        final_pwm_hs_v_out = 1'b0; final_pwm_ls_v_out = 1'b1;
64                        final_pwm_hs_w_out = 1'b0; final_pwm_ls_w_out = 1'b1;
65                    end
```

Above: code snippet, below: simulation on GTKWave



This waveform simulation clearly illustrates the correct operation of the brake_logic module, showcasing its ability to override normal motor control signals and apply different, software-defined braking patterns. Initially, the module is in a non-braking state, where the final_pwm_*_out signals (in yellow) are simply passing through the incoming pwm_*_in signals (in red), demonstrating normal motor operation. At approximately 1.2 µs, the sw_brake_enable_in_tb signal is asserted high, causing the final_brake_active signal to go high and initiating the first braking phase. As the current_brake_pattern is set to 2'b01, the module immediately overrides the outputs, forcing all High-Side outputs low and all Low-Side outputs high, which corresponds to the dynamic braking mode. This state is held until the event highlighted by the pink cursor at 5.51 µs. At this precise moment, while the brake remains active, the current_brake_pattern is changed from 2'b01 to 2'b00. The logic responds instantly, transitioning from the dynamic brake to the "coast" brake mode by forcing all six final PWM outputs to a low state. This successful, clean transition between two distinct braking strategies while the main brake remains engaged confirms the module's hierarchical control logic is functioning perfectly, providing a robust and flexible safety system for the motor controller.

HDL TESTBENCH: TESTING THE HARDWARE’S CODE

Prior to integration within the full System-on-Chip (SoC) environment, a critical hardware-level verification phase was undertaken to validate the functional correctness of the BLDC controller IP in isolation. This was accomplished by developing a dedicated Verilog testbench designed to instantiate the bldc_controller module as the Device Under Test (DUT). By directly manipulating the inputs for clock, reset, and bus commands, the testbench could systematically write to the controller's memory-mapped registers, thereby commanding the DUT to enter each of its designed operational modes. This foundational step was essential for focused debugging of the HDL code, allowing us to

isolate and resolve any logic errors within the peripheral itself, free from the added complexities of the processor, toolchain, and C code.

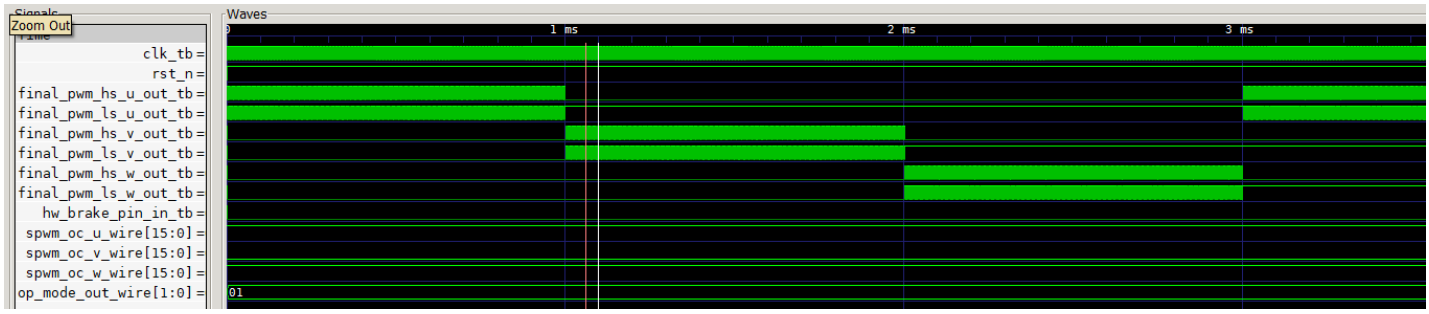
The testbench was scripted to cycle through a sequence of configurations, and the resulting behavior of the `final_pwm` output signals was meticulously inspected.

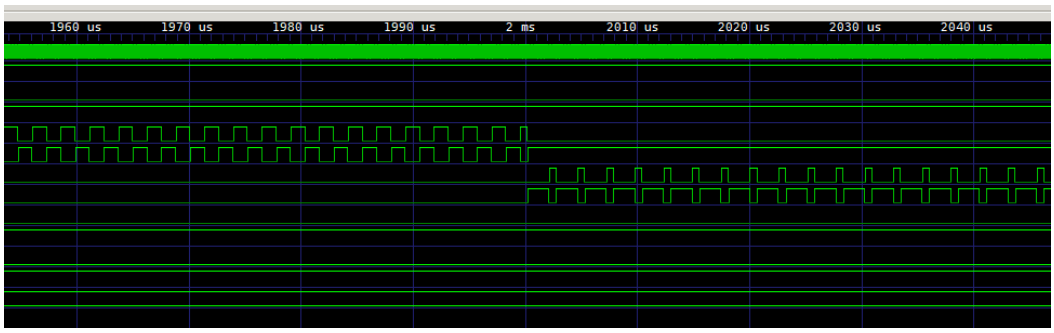
- *For **Static Mode**, this involved verifying that the correct high-side outputs were held active according to the six-step commutation sequence.*



- *In 6-step **PWM Mode**, verification focused on observing the constant-duty-cycle pulses on the active phase.*

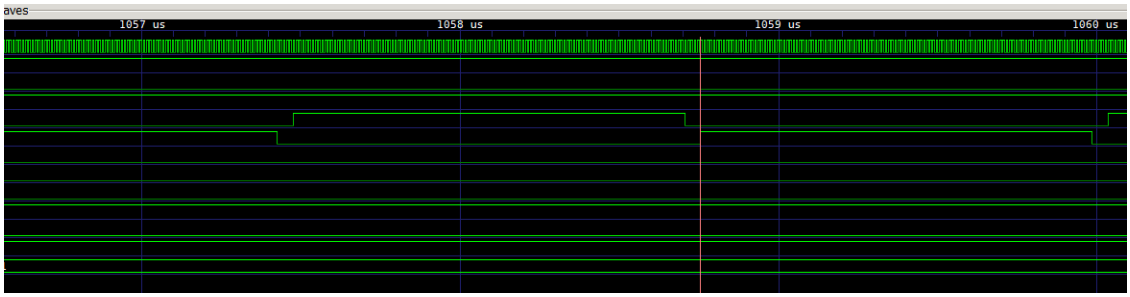
The first picture shows the channels activating separately, and the second one, zooming closely, shows how in pwm the signals go high and low repeatedly based on the duty cycle.





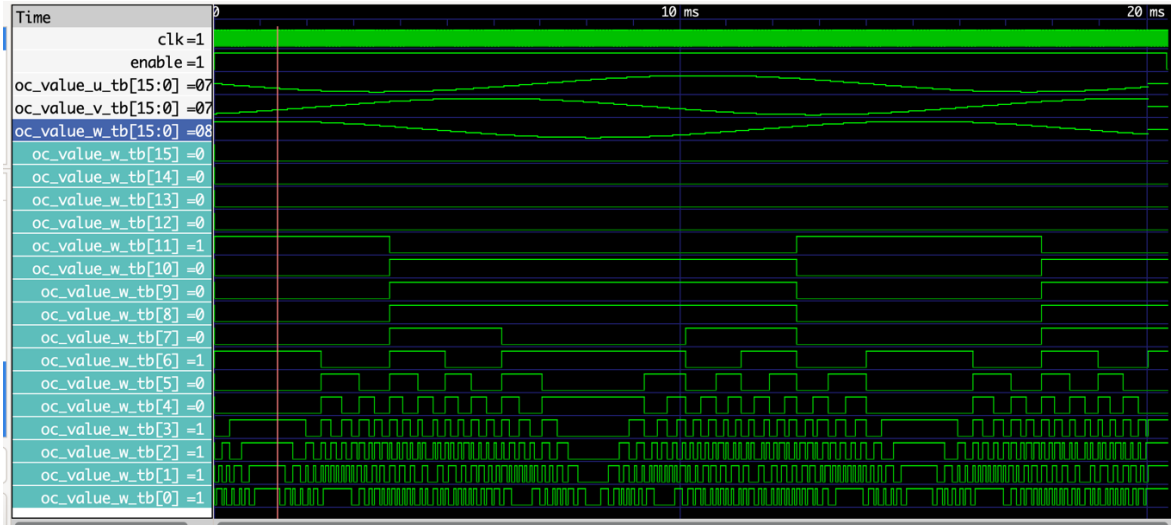
Also, the last signals of the GTKwave, the “spwm” ones, show a small piece of the sinusoidal wave that can drive the motor in the SPWM mode. Further explanation will follow in the next pages.

Looking even more closely in those signals, it’s also noticeable the dead-zone implementation:

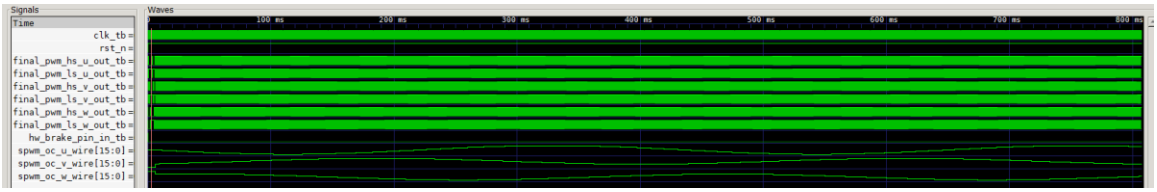


The most rigorous validation was for SPWM mode, where the sinusoidal envelope formed by the varying pulse widths and the correct 120-degree phase shift between the U, V, and W outputs were confirmed.

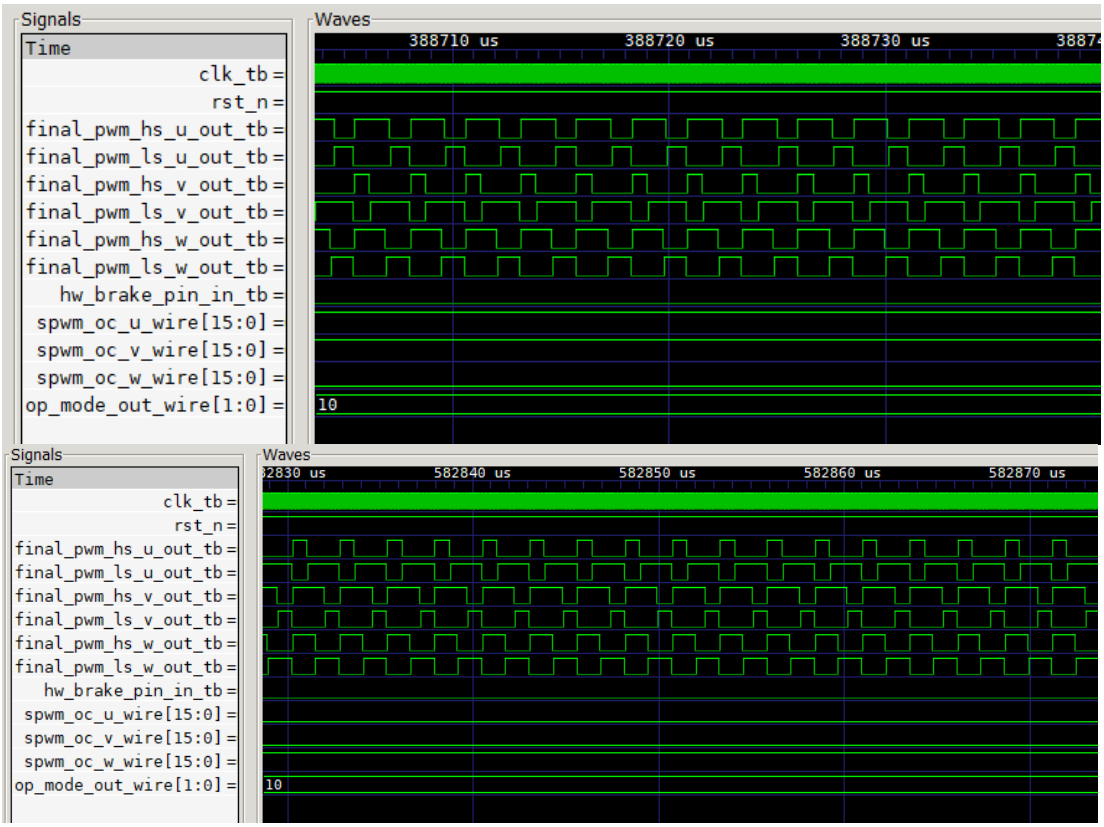
The first picture shows the signals coming from the spwm_engine.v module.



The second picture shows the three final channels of the motor, acting all together.



Looking closely on the third and fourth picture, it’s possible to notice the 120° shift mentioned above. The two pictures are separated for clarity reasons, since it was difficult to show clearly the shifting of the signals in one single screenshot.



The success of this hardware simulation gave us high confidence that the Verilog implementation was robust and ready for integration into the full system.

CPU: HUMMINGBIRD E203

The Hummingbird E203 is a 32-bit RISC-V processor core developed by Nuclei System Technology, specifically optimized for low-power and area-efficient embedded applications. It implements the RV32IMAC instruction set architecture, which includes the base integer instruction set (I), extensions for integer multiplication and division (M), atomic instructions (A), and 16-bit compressed instructions (C) for improved code density (No, it’s not exclusive for apple computers :)). Architecturally, the E203 core features a simple yet effective 2-stage pipeline (Fetch and Execute), which minimizes resource utilization and power consumption, making it ideal for FPGA implementation. Within the context of this project, the E203 acts as the central master of the SoC, responsible for executing the C-language control program. It communicates with all peripherals, including our custom BLDC driver, through the dedicated Internal Chip Bus (ICB), issuing read and write commands to their memory-mapped registers to orchestrate the overall system behavior.

INTERNAL BUS

Communication between the Hummingbird E203 CPU core and its peripherals is handled by the Internal Chip Bus (ICB), a lightweight, proprietary interface designed for efficiency within the SoC. The ICB architecture is based on a simple two-channel structure: a **Command Channel** and a **Response Channel**. The Command Channel is used by the master (the CPU) to send an address, write data, and control signals to a selected slave (such as our BLDC driver). The Response Channel is used by the slave to return read data or response information back to the master. Data transfers on both channels are

governed by a `valid/ready` handshake protocol, which ensures synchronized communication. In the context of this project, the ICB is the essential data pathway that allows the C program running on the CPU to control the motor by writing configuration parameters to and reading status from the memory-mapped registers of our custom BLDC driver peripheral.

CONNECTING THE BLDC DRIVER TO THE SYSTEM BUS

To enable communication between the E203 CPU and the custom BLDC driver, the driver must be integrated as a new peripheral slave onto the Internal Chip Bus (ICB). This integration is performed within the `e203_subsys_perips.v` file, which serves as the central hub for all SoC peripherals. The connection process involves several key steps: first, the driver is assigned a unique base address in the system's memory map by configuring a free slot in the ICB bus arbiter.

```
1683  //this is the address of our module:
1684  //  * BLDC0     : 0x1001 4000 -- 0x1001 4FFF
1685  .05_BASE_ADDR       (32'h1001_4000),
1686  .05_BASE_REGION_LSB (12),
1687
1688
1689  //-----------------------------------------------------------------------
1690  //-----------------------------------------------------------------------
1691  //-----------------------------------------------------------------------
1692  //  * PWM0      : 0x1001 5000 -- 0x1001 5FFF
1693  .06_BASE_ADDR       (32'h1001_5000),
1694  .06_BASE_REGION_LSB (12),
1695  //  * UART1     : 0x1002 3000 -- 0x1002 3FFF
1696  .07_BASE_ADDR       (32'h1002_3000),
1697  .07_BASE_REGION_LSB (12),
```

This gives the peripheral a distinct address range.

Next, we must declare the wires we’re going to use and instantiate the BLDC motor top module:

e203_hello_world-main > rtl > core > e203_subsys_perips.v

```
1004  //-----------------------------------------------------------------------
1006   //wire declarations
1007   // duty 10ms update
1008
1009   wire                  BLDC_driver_i_icb_cmd_valid;
1010   wire                  BLDC_driver_i_icb_cmd_ready;
1011   wire       [31:0]     BLDC_driver_i_icb_cmd_addr;
1012   wire                  BLDC_driver_i_icb_cmd_read;  // 1 for read, 0 for write
1013   wire       [31:0]     BLDC_driver_i_icb_cmd_wdata;
1014
1015   wire                  BLDC_driver_i_icb_rsp_valid;
1016   wire                  BLDC_driver_i_icb_rsp_ready; // Master ready to accept response
1017   wire       [31:0]     BLDC_driver_i_icb_rsp_rdata;
1018
1019   wire     BLDC_driver_io_pad_out_hsu; // HS_U
1020   wire     BLDC_driver_io_pad_out_lsu; // LS_U
1021   wire     BLDC_driver_io_pad_out_hsv; // HS_V
1022   wire     BLDC_driver_io_pad_out_lsv; // LS_V
1023   wire     BLDC_driver_io_pad_out_hsw; // HS_W
1024   wire     BLDC_driver_io_pad_out_lsw; // LS_W
1025
1026   wire     BLDC_driver_io_pad_in_BREAK; // Emergency break signal
```

```
2714  //our instantiation of the module:
2715  bldc_controller u_BLDC_driver_top(
2716    .clk (clk),
2717    .rst_n(rst_n),
2718    .icb_cmd_valid (BLDC_driver_i_icb_cmd_valid),
2719    .icb_cmd_ready (BLDC_driver_i_icb_cmd_ready),
2720    .icb_cmd_addr (BLDC_driver_i_icb_cmd_addr),
2721    .icb_cmd_read (BLDC_driver_i_icb_cmd_read),
2722    .icb_cmd_wdata (BLDC_driver_i_icb_cmd_wdata),
2723
2724    .icb_rsp_valid (BLDC_driver_i_icb_rsp_valid),
2725    .icb_rsp_ready (BLDC_driver_i_icb_rsp_ready),
2726    .icb_rsp_rdata (BLDC_driver_i_icb_rsp_rdata),
2727
2728    .hw_brake_pin_in (BLDC_driver_io_pad_in_BREAK),   // Emergency break signal
2729    //.brake_interrupt_out (BLDC_irq_wire),
2730
2731    // Motor Control Outputs (to H-Bridge driver)
2732    .final_pwm_hs_u_out (BLDC_driver_io_pad_out_hsu),
2733    .final_pwm_ls_u_out (BLDC_driver_io_pad_out_lsu),
2734    .final_pwm_hs_v_out (BLDC_driver_io_pad_out_hsv),
2735    .final_pwm_ls_v_out (BLDC_driver_io_pad_out_lsv),
2736    .final_pwm_hs_w_out (BLDC_driver_io_pad_out_hsw),
2737    .final_pwm_ls_w_out (BLDC_driver_io_pad_out_lsw)
2738
2739    // Inputs from Motor/External if we wanted hall sensors
2740    /*.io_pad_in_HAL_A (BLDC_driver_io_pad_in_HAL_A),    // Hall sensor A
2741    .io_pad_in_HAL_B (BLDC_driver_io_pad_in_HAL_B),    // Hall sensor B
2742    .io_pad_in_HAL_C (BLDC_driver_io_pad_in_HAL_C)     // Hall sensor C */
2743  );
```

```
2747  //assign the new output value
2748  //assign perips_irq_2_plic_BLDC = BLDC_irq_wire;
2749  assign BLDC_driver_out_hsu = BLDC_driver_io_pad_out_hsu;
2750  assign BLDC_driver_out_lsu = BLDC_driver_io_pad_out_lsu;
2751  assign BLDC_driver_out_hsv = BLDC_driver_io_pad_out_hsv;
2752  assign BLDC_driver_out_lsv = BLDC_driver_io_pad_out_lsv;
2753  assign BLDC_driver_out_hsw = BLDC_driver_io_pad_out_hsw;
2754  assign BLDC_driver_out_lsw = BLDC_driver_io_pad_out_lsw;
2755  assign BLDC_driver_in_BREAK = BLDC_driver_io_pad_in_BREAK;
```

```
1857   //this is where the ICB signals of
1858   // our module connect with the bus
1859   //  * BLDC0
1860     .o5_icb_enable     (1'b1),
1861
1862     .o5_icb_cmd_valid  (BLDC_driver_i_icb_cmd_valid),
1863     .o5_icb_cmd_ready  (BLDC_driver_i_icb_cmd_ready),
1864     .o5_icb_cmd_addr   (BLDC_driver_i_icb_cmd_addr),
1865     .o5_icb_cmd_read   (BLDC_driver_i_icb_cmd_read),
1866     .o5_icb_cmd_wdata  (BLDC_driver_i_icb_cmd_wdata),
1867     .o5_icb_cmd_wmask  (),
1868     .o5_icb_cmd_lock   (),
1869     .o5_icb_cmd_excl   (),
1870     .o5_icb_cmd_size   (),
1871     .o5_icb_cmd_burst  (),
1872     .o5_icb_cmd_beat   (),
1873
1874     .o5_icb_rsp_valid  (BLDC_driver_i_icb_rsp_valid),
1875     .o5_icb_rsp_ready  (BLDC_driver_i_icb_rsp_ready),
1876     .o5_icb_rsp_err    (1'b0  ),
1877     .o5_icb_rsp_excl_ok(1'b0  ),
1878     .o5_icb_rsp_rdata  (BLDC_driver_i_icb_rsp_rdata),
```

Now, the driver's ICB slave interface ports (`i_icb_cmd_*`, `i_icb_rsp_*`) are wired directly to the corresponding signals of the assigned arbiter slot. This process effectively memory-maps the driver's registers, making them accessible to the CPU through standard load and store instructions and allowing software to configure and control the motor.

GPIO ASSIGNMENTS

To control the motor and receive the inputs, the driver must be connected to the GPIOs(general purpose input/output pins), which are the main way for signal exchanging.

```
1034   //cerca BLDC driver controller GPIO assignment
1035   // This part configures the physical I/O pads for the FPGA.
1036   // for each pin, we define its direction (input/output) and
1037   // physical properties
1038   // --- BLDC Driver Outputs (to Motor Driver Board) ---
1039   // High-Side U-Phase -> GPIO 26
1040   assign io_pads_gpio_26_o_oval = BLDC_driver_io_pad_out_hsu;
1041   assign io_pads_gpio_26_o_oe   = 1'b1;   // Output Enable: ON
1042   assign io_pads_gpio_26_o_ie   = 1'b0;   // Input Enable: OFF
1043   assign io_pads_gpio_26_o_pue  = 1'b0;   // Pull-Up Enable: OFF
1044   assign io_pads_gpio_26_o_ds   = 1'b0;   // Drive Strength: Default
1045
1046   // Low-Side U-Phase -> GPIO 27
1047   assign io_pads_gpio_27_o_oval = BLDC_driver_io_pad_out_lsu;
1048   assign io_pads_gpio_27_o_oe   = 1'b1;
1049   assign io_pads_gpio_27_o_ie   = 1'b0;
1050   assign io_pads_gpio_27_o_pue  = 1'b0;
1051   assign io_pads_gpio_27_o_ds   = 1'b0;
1052
1053   // High-Side V-Phase -> GPIO 28
1054   assign io_pads_gpio_28_o_oval = BLDC_driver_io_pad_out_hsv;
1055   assign io_pads_gpio_28_o_oe   = 1'b1;
1056   assign io_pads_gpio_28_o_ie   = 1'b0;
1057   assign io_pads_gpio_28_o_pue  = 1'b0;
1058   assign io_pads_gpio_28_o_ds   = 1'b0;
```

```
// Low-Side V-Phase -> GPIO 29
assign io_pads_gpio_29_o_oval = BLDC_driver_io_pad_out_lsv;
assign io_pads_gpio_29_o_oe   = 1'b1;
assign io_pads_gpio_29_o_ie   = 1'b0;
assign io_pads_gpio_29_o_pue  = 1'b0;
assign io_pads_gpio_29_o_ds   = 1'b0;

// High-Side W-Phase -> GPIO 30
assign io_pads_gpio_30_o_oval = BLDC_driver_io_pad_out_hsw;
assign io_pads_gpio_30_o_oe   = 1'b1;
assign io_pads_gpio_30_o_ie   = 1'b0;
assign io_pads_gpio_30_o_pue  = 1'b0;
assign io_pads_gpio_30_o_ds   = 1'b0;

// Low-Side W-Phase -> GPIO 31
assign io_pads_gpio_31_o_oval = BLDC_driver_io_pad_out_lsw;
assign io_pads_gpio_31_o_oe   = 1'b1;
assign io_pads_gpio_31_o_ie   = 1'b0;
assign io_pads_gpio_31_o_pue  = 1'b0;
assign io_pads_gpio_31_o_ds   = 1'b0;

//the brake is an input and not an output
// Emergency Brake Input -> GPIO 23
assign BLDC_driver_io_pad_in_BREAK = io_pads_gpio_23_i_ival;
assign io_pads_gpio_23_o_oval   = 1'b0;
assign io_pads_gpio_23_o_ie     = 1'b1;
assign io_pads_gpio_23_o_oe     = 1'b0;
assign io_pads_gpio_23_o_pue    = 1'b0;
assign io_pads_gpio_23_o_ds     = 1'b0;
```

The brake pin is the only one that works as an input, it is in fact the pin being assigned to the brake value and not vice versa.

The gpio are by default connected to the top module of the system e230_soc_demo.v.

THE C CODE

Now we must develop the software to drive our hardware, let's see the steps to make it together.

First of all, take note that all modifications are being done inside the folder "firmware/BLDCfaf/src". The C code's core consists of two files: main.c and bldc_driver.h, located in "firmware/BLDCfaf/src". Before talking about the content of these two files, we must consider the files that must be included inside these two, we are talking about the platform.h and init.h files. They are already part of the e203 files and can be found as shown here. Make sure to include these files in the C files you will use.

```
7    #include "bsp/hbird-e200/env/platform.h"
8    #include "bsp/hbird-e200/env/init.h"
```

Especially the platform.h file is very important because it will include information about the base address in memory where the instructions for the motor will be stored.

```
44   #define BLDC_DRIVER_CTRL_ADDR     _AC(0x10014000,UL)
```

```
123  #define BLDC_REG(offset)        _REG32(BLDC_DRIVER_CTRL_ADDR, offset)
```

A big problem I encountered was that the platform.h file itself includes many more files that couldn't easily be found by the compiler. We are talking about further files included in the e203 project that describe some standard parameters for the CPU.

```
#include "devices/const.h"
#include "devices/aon.h"
#include "devices/clint.h"
#include "devices/gpio.h"
#include "devices/plic.h"
#include "devices/pwm.h"
#include "devices/spi.h"
#include "devices/uart.h"
#include "devices/i2c.h"
```

These files were however included in a different branch of the directory tree:

firmware\BLDCfaf\src\bsp\hbird-e200\include\headers

Such folder was however tricky to reach, as a result compilation generated the following error:

fatal error: headers/const.h: No such file or directory

#include "headers/const.h"

^~~~~~~~~~~~~~~~~~

compilation terminated.

Two primary strategies were attempted to resolve the issue:

1. **Modifying platform.h to Adjust the Include Paths:**
   We edited the include directives in platform.h to use fully relative paths or to correct the subdirectory naming, ensuring that they accurately reflected the file hierarchy. Despite being a theoretically clean approach, it proved ineffective in isolation, as the compiler still lacked the correct context unless provided via a build system.
2. **Configuring IntelliSense and Compiler Include Paths:**
   We added the correct include path to the c_cpp_properties.json file for IntelliSense support, and attempted to configure a proper build task in .vscode/tasks.json to pass the required -I flag to gcc. Although this resolved IntelliSense warnings, compilation

continued to fail due to the build system not consistently using the configured include paths.

After these ineffective operations, a more pragmatic attempt was made.

Duplicating Header Files

Due to time constraints and the need for a working build environment, we **duplicated all required header files** (const.h, gpio.h, plic.h…) from their original location in the "include/headers" directory to the "env" directory, where platform.h resides.

This resolved the issue because it allowed the compiler to find the included files without needing path adjustments or compiler flags. Since these headers are **standard, vendor-supplied files specific to the Hummingbird E203 core** and **are not expected to change** over the course of our development, duplication poses no risk of inconsistency in our project's context.

This allowed us to progress without introducing unnecessary complexity into the build. Moving forward, this decision can be revisited if modifications to those header files become necessary or the project scales in complexity.

BLDC_DRIVER.H

The bldc_driver.h header file serves as the critical link between the high-level C software running on the E203 processor and the low-level Verilog hardware of the BLDC motor controller peripheral, allowing the software to interact with the hardware registers in a structured and error-resistant manner. This is achieved by defining a complete and human-readable map of the peripheral's six 32-bit registers.

The file begins by defining the memory offset for each of the six registers relative to the peripheral's base address, which is established in platform.h.

```
19   #define REG_CONTROL_OFFSET              0x00
20   #define REG_CARRIER_TIMING_OFFSET       0x04
21   #define REG_COMMUTATION_TIMING_OFFSET   0x08
22   #define REG_PWM_DUTY_OFFSET             0x0C
23   #define REG_SPWM_CONFIG_OFFSET          0x10
24   #define REG_STATUS_OFFSET               0x14
```

It then creates easy-to-use macros (CONTROL_REG, STATUS_REG…) that allow the C code to access these hardware locations as if they were simple variables. The core of the file is the detailed breakdown of each register into its constituent bitfields. For every configurable parameter, such as OP_Mode or DEAD_ZONE_VALUE, two constants are defined: a _BIT constant specifying the starting bit (LSB) of the field, and a _MASK constant defining its width. For instance, a 2-bit field would have a mask of 0x3 (binary 11), while a 16-bit field would have a mask of 0xFFFF.

```
67   #define REG1_DEAD_ZONE_VALUE_BIT        16  // [19:16]
68   #define REG1_DEAD_ZONE_VALUE_MASK       0xF // 4 bits
```

These masks are essential for performing safe read-modify-write operations, enabling the software to precisely modify a specific setting within a register without corrupting any adjacent data. Finally, the header includes enumerations (enum) for multi-bit fields like operating modes and brake patterns, replacing raw numbers with descriptive names (e.g., MODE_SPWM_SINE) to make the main C code more intuitive and less prone to errors.

MAIN.C

To validate the functionality of the Verilog hardware peripheral, a series of C programs were developed to run on the E203 soft-core

processor. Rather than creating a single, monolithic test program, we adopted a strategy of writing multiple, targeted main.c files. Each file is designed to test a specific sequence of operations, allowing for focused and incremental verification of the controller's features. This approach ensures that simulations remain concise and manageable; each test can be compiled and run independently to verify a particular aspect of the hardware before moving on to more complex sequences.

The test programs were designed to be fully autonomous, utilizing simple software delay loops (simple_delay) to sequence through different operational stages without requiring external user input. The test sequences were crafted to cover all major operating modes of the controller. Multiple tests have been made to test specific functionalities only a few at a time, one only for static and brakes, a subsequent more complex transits from static onto PWM mode with a fixed 50% duty cycle and a final, more comprehensive with also the SPWM mode.

The tests have so been gradual, and the results will be shown in the following.

FIRMWARE DESIGN AND VERIFICATION

The firmware developed for this project serves as the intelligent control layer, running on the embedded RISC-V CPU to configure and command the custom BLDC motor controller peripheral.

Given the computational intensity of full-system co-simulation, which limited the practical duration of each test run, a single, integral test program was too large to run. Therefore, a series of targeted C applications were created. This strategic partitioning of tests was essential; each application was designed to isolate and thoroughly validate a specific operating mode—Static, 6-Step PWM, or Sinusoidal PWM.

C CODE STRUCTURE AND PRINCIPLES

The verification firmware was written in C.

In the codes, all register access macros are defined using the volatile keyword, a crucial directive that prevents the compiler from making incorrect optimizations by assuming the register's value cannot change on its own, which is fundamental for memory-mapped I/O.

Each test program follows a canonical, multi-stage execution flow designed for clarity and repeatability. The main() function begins with system initialization, followed by a hardware reset sequence that clears all peripheral registers and triggers a soft reset to ensure the Verilog state machines begin in a known, default state. The code then enters a configuration stage where local C variables are used to define key operational parameters, such as motor speed or duty cycle. The core of the program involves programming the hardware by writing these parameters to the appropriate registers.

This is achieved through direct, low-level read-modify-write operations. To set or clear individual bits within a register without disturbing its other fields, bitwise logic is employed (|= with a bitmask to set, &= ~ to clear), which is a mandatory practice for safely controlling complex hardware.

```
62      // Configure control register for SPWM mode and direction
63      temp_reg_val = 0;
64      temp_reg_val |= (MODE_SPWM_SINE << REG0_OP_MODE_BIT);
65      temp_reg_val |= (1U << REG0_DIRECTION_BIT); // Set Direction to Clockwise
66      CONTROL_REG = temp_reg_val;
67      printf(" -> Set Mode to SPWM_SINE, Direction to CW. CONTROL_REG = 0x%08X\n", (unsigned int)CONTROL_REG);
```

```
81      // CRITICAL: Disable motor drive before enabling brake
82      CONTROL_REG &= ~(1U << REG0_ENABLE_MOTOR_BIT);
83      printf(" -> Motor drive disabled.\n");
```

Once configured, the motor is enabled, and a simple_delay() function—a "busy-wait" loop using nop instructions—is called. This function creates a predictable software pause, providing a stable

window of time during which the hardware's behavior can be observed and analyzed in the simulation waveform. Finally, the test concludes by disabling the motor and entering an infinite idle loop.

```
// Let it run for a while to see several full sine wave rotations
simple_delay(600000);
```

STATIC AND 6-STEP PWM MODE FIRMWARE

The firmware for testing the Static and 6-Step PWM modes is nearly identical in structure, reflecting the fact that both modes rely on the same underlying 6-step commutation finite-state machine (FSM) in the Verilog hardware.

The Static Mode test represents the simplest case, designed to validate the fundamental machinery of the controller: bus communication, register write/read capability, and the basic commutation FSM logic. The software's only tasks are to select the mode and set the COMMUTATION_TIMING_REG to define the rotation speed.

The 6-Step PWM test builds directly upon this foundation. The C code is identical, with the crucial addition of configuring two more hardware parameters: the PWM carrier frequency is set in the CARRIER_TIMING_REG, and a fixed duty cycle is written to the PWM_DUTY_REG. This minimal software change commands the hardware to engage its PWM generation logic for the active phase, demonstrating a clear and logical progression of feature verification.

```
    // Define parameters for this mode
    uint32_t temp_reg_val = 0;
    uint16_t dead_zone_val = 5;
    uint16_t pwm_carrier_period = 1000;
    uint16_t duty_50_percent = pwm_carrier_period / 2;
    uint16_t desired_comm_period = 30000; // This controls the 6-step rotation speed

    // Configure timing registers
    CARRIER_TIMING_REG = (dead_zone_val << REG1_DEAD_ZONE_VALUE_BIT) | (pwm_carrier_period << REG1_PWM_CARRIER_PERIOD_BIT);
    printf(" -> Set Dead Zone, Carrier Period. CARRIER_TIMING_REG = 0x%08X\n", (unsigned int)CARRIER_TIMING_REG);

    COMMUTATION_TIMING_REG = (desired_comm_period << REG2_COMMUTATION_STEP_PERIOD_BIT);
    printf(" -> Set Commutation Period. COMMUTATION_TIMING_REG = 0x%08X\n", (unsigned int)COMMUTATION_TIMING_REG);

    PWM_DUTY_REG = (duty_50_percent << REG3_PWM_DUTY_VALUE_GENERIC_BIT);
    printf(" -> Set PWM Duty Value. PWM_DUTY_REG = 0x%08X\n", (unsigned int)PWM_DUTY_REG);

    // Configure control register for PWM mode and direction
    temp_reg_val = 0;
    temp_reg_val |= (MODE_PWM_6STEP << REG0_OP_MODE_BIT);
    temp_reg_val |= (1U << REG0_DIRECTION_BIT); // Set Direction to Clockwise
    CONTROL_REG = temp_reg_val;
    printf(" -> Set Mode to PWM_6STEP, Direction to CW. CONTROL_REG = 0x%08X\n", (unsigned int)CONTROL_REG);
```

SINUSOIDAL PWM (SPWM) MODE FIRMWARE

In the firmware for testing SPWM mode the software no longer concerns itself with the low-level details of commutation timing or fixed duty cycles. Its role becomes more abstract: it commands a desired *behavior*.

The C code's primary responsibility is to configure the SPWM_CONFIG_REG with two high-level parameters: spwm_amplitude, which sets the target voltage by controlling the peak of the sine wave, and spwm_phase_increment. This phase increment value is the main and key software input to the hardware's Numerically Controlled Oscillator (NCO), giving the CPU direct control over the sine wave's electrical frequency. Thus, the motor's rotational speed. By simply changing this single value in software, the firmware can command a wide range of motor speeds, while the spwm_engine in the Verilog hardware handles all the complex real-time calculations required to generate the three phase-shifted sinusoidal waveforms.

```
    // Define parameters for this mode
    uint32_t temp_reg_val = 0;
    uint16_t dead_zone_val = 5;
    uint16_t pwm_carrier_period = 1000;
    uint16_t spwm_phase_increment = 10;  // Controls motor speed (electrical frequency)
    uint8_t  spwm_amplitude = 200;       // Controls voltage/torque (max 255)

    // Configure timing registers (SPWM uses the same PWM carrier)
    CARRIER_TIMING_REG = (dead_zone_val << REG1_DEAD_ZONE_VALUE_BIT) | (pwm_carrier_period << REG1_PWM_CARRIER_PERIOD_BIT);
    printf(" -> Set Dead Zone, Carrier Period. CARRIER_TIMING_REG = 0x%08X\n", (unsigned int)CARRIER_TIMING_REG);

    // Configure the SPWM-specific register
    SPWM_CONFIG_REG = (spwm_amplitude << REG4_SPWM_AMPLITUDE_GLOBAL_BIT) | (spwm_phase_increment << REG4_SPWM_PHASE_INCREMENT_BIT);
    printf(" -> Set SPWM Amp, Phase Inc. SPWM_CONFIG_REG = 0x%08X\n", (unsigned int)SPWM_CONFIG_REG);

    // Configure control register for SPWM mode and direction
    temp_reg_val = 0;
    temp_reg_val |= (MODE_SPWM_SINE << REG0_OP_MODE_BIT);
    temp_reg_val |= (1U << REG0_DIRECTION_BIT); // Set Direction to Clockwise
    CONTROL_REG = temp_reg_val;
    printf(" -> Set Mode to SPWM_SINE, Direction to CW. CONTROL_REG = 0x%08X\n", (unsigned int)CONTROL_REG);

    // Enable the motor
    CONTROL_REG |= (1U << REG0_ENABLE_MOTOR_BIT);
    printf(" -> Motor ENABLED in SPWM mode. Running for an extended duration...\n");

    // Let it run for a while to see several full sine wave rotations
    simple_delay(600000);
```

All files have been compiled on Visual Studio Code to ensure no syntax errors were present and that the necessary header files were correctly included. Running the compiled file will obviously lead to failure and errors, because such C files need to be connected to the whole system, including all e203 files and Verilog files. To try them out, it is hence necessary to generate machine code out of them.

FINAL TESTING

RUNNING THE MAKE COMMAND AND TESTING

So… the machine code! We must open the “firmware/bldcfaf/debug” folder and select “open terminal” after right-clicking. We can then run the command “make” (“make clean” is useful to clear the old files) to generate all that is needed in a file called ram.hex.

We then loaded all our Verilog modules inside "rtl/ip" and now we are finally set up for the grand test.

To run a simulation inside Visual Studio Code, we used the following commands:

Inside the terminal of *e203_BLDCfaf-main*>, we must write:

ls (shows the files present in your directory ->useful to check before and after commands)

cd sim (moves into the folder "sim")

ls

.\sim_sys_tb.cmd (Runs simulation script in Windows syntax)

The simulation above includes a testbench for the SoC that instantiates the top module of the e203 (e203_soc_demo)

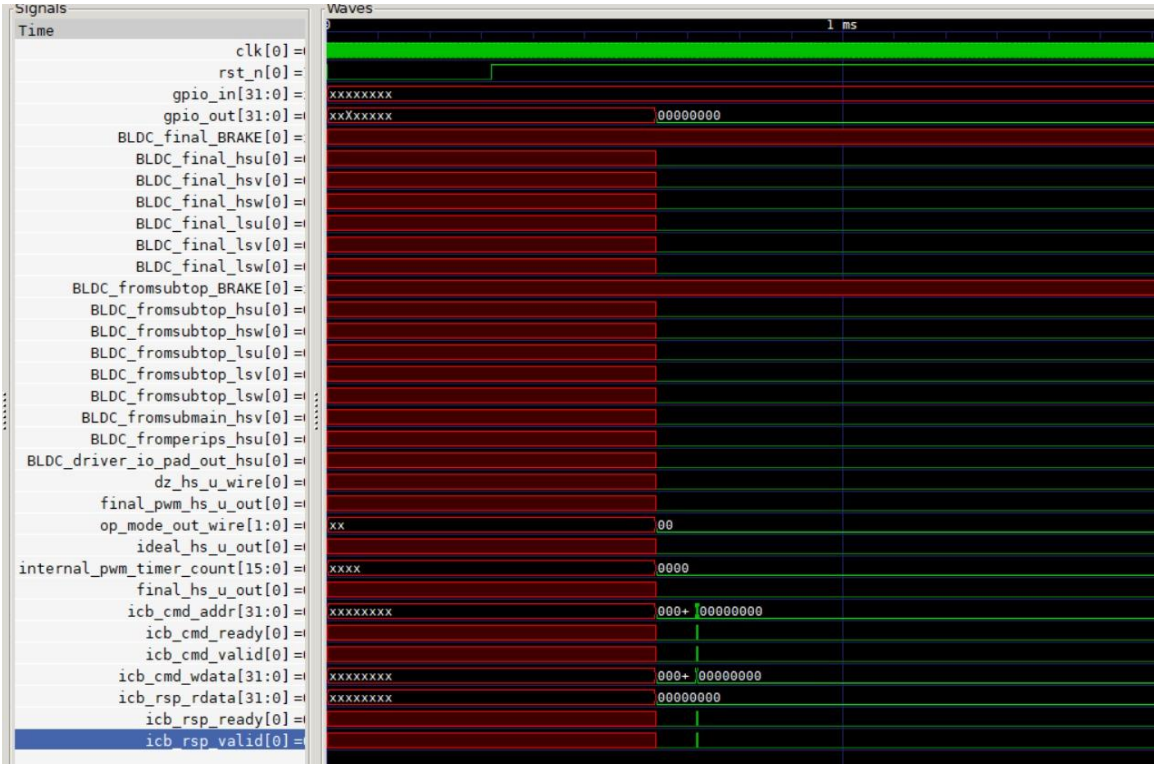
ls

vvp -n wave.out -lxt2 (Run a compressed compiled simulation)

The simulation will need from some tens of minutes to a couple of hours, do not touch the terminal until it is done. When it is, you will see a wave form file weighing up to a few hundred megabytes. After that, through

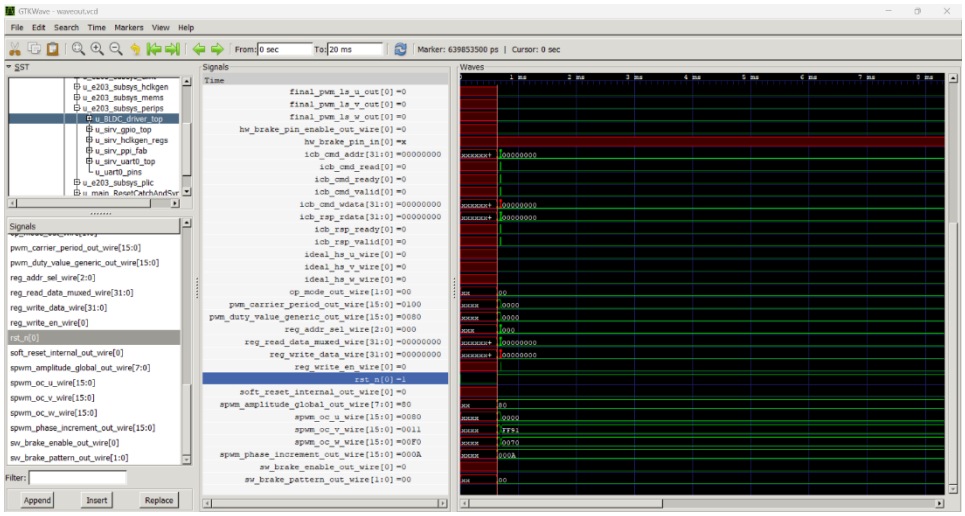
gtkwave waveforms.out (or whatever the name of you wave file is)

you can open the GTKWave to view the simulation. You must navigate to "…/u_e203_subsys_perips/u_BLDC_driver_top" to view the project's waves. Our first simulations led to *mildly* disappointing results.



As you can see, our first test was greatly unsuccessful with all the signals being constant zeroes and the only traces of activity were around the successful handshake.

After going through all our code again, checking that the reset values were coherent in all files, fixing some typos and making some brand-new C code, here comes the next test.



The second test led to a similar outcome, confirming the issue was likely in the software's interaction with the bus rather than the hardware modules themselves. Digging deeper from that simulation we arrived to the final solution, that is the C code structure described above.

Now, to thoroughly validate the distinct operating modes of the BLDC motor controller we had to run the final simulations.

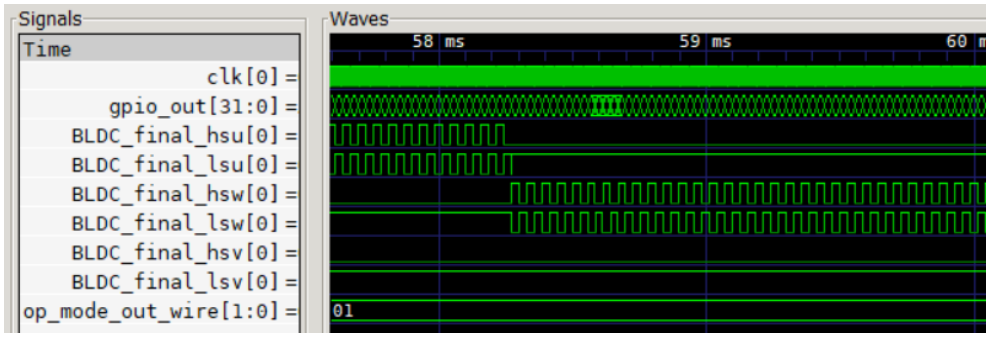
Due to the complexity of the full SoC we used a compressed format (vvp *-n* wave.out *-lxt2*) to compile the simulation, and because of that, the waveforms could not run for longer than 100ms. Therefore, the final system verification was conducted through a series of targeted simulations, dedicating one for each operating mode, allowing for a clearer and detailed analysis.

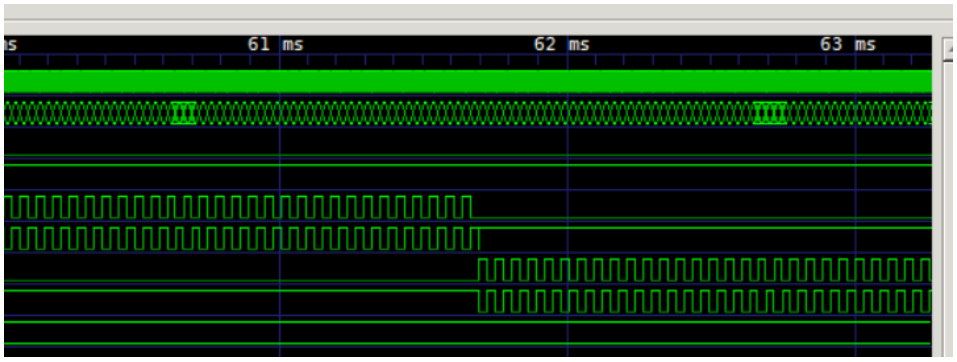
STATIC MODE VERIFICATION

The initial simulation focused on verifying the controller’s Static Mode. The primary goal was to confirm fundamental CPU-to-peripheral communication over the ICB bus and the correct operation of the 6-step commutation logic. In this test, the C program, running on the RISC-V core, wrote configuration values to the peripheral's control registers to select Static Mode, set a motor direction, and define a commutation step period. The motor was then enabled. The resulting waveform successfully demonstrated that the active high-side output for a given phase was held solid HIGH, while the corresponding low-side FETs were enabled on the other phases to create a current path. The simulation clearly shows the hardware periodically advancing through the six commutation states, with the timing of each step correctly governed by the step_timer module. This test successfully validated the core bus interface, the register file, and the state transition logic.

PWM MODE VERIFICATION

The second simulation was designed to verify the functionality of 6-Step PWM Mode. This test built upon the first by introducing the PWM carrier signal. The C code configured the controller for PWM mode and set a fixed 50% duty cycle via the PWM_DUTY_REG. The key difference observed from Static mode was that the active high-side output was being pulsed at a high frequency corresponding to the configured PWM carrier period. The width of these pulses matched the 50% duty cycle value commanded by the software.

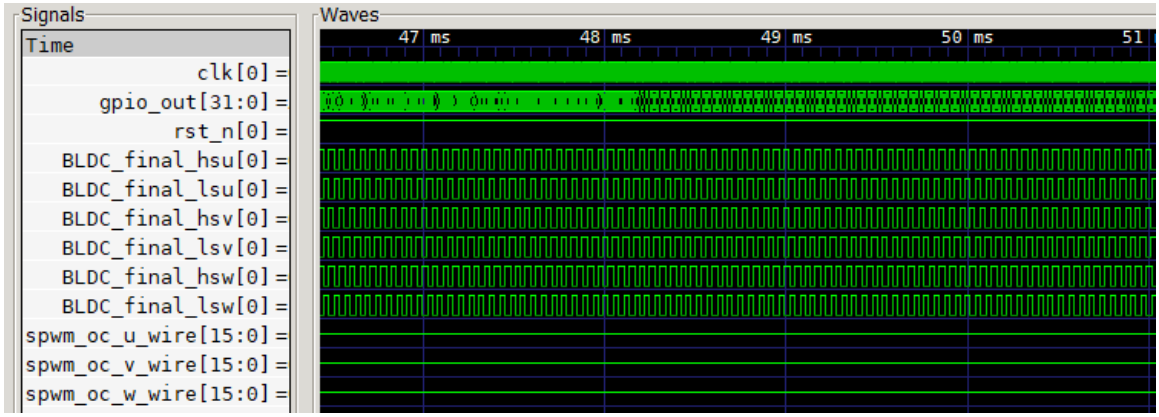




SINUSOIDAL PWM (SPWM) MODE VERIFICATION

The final and most complex test validated the Sinusoidal PWM (SPWM) mode. The C program configured the controller for SPWM mode and wrote values for phase increment (motor speed) and amplitude (voltage) to the SPWM_CONFIG_REG. Due to the same issue described above (the simulation being short for practical limitations), the running time is not sufficient to show a clear change of duty cycle.

Nonetheless, we can confidently suppose that the simulation would be working properly if we had a longer timeline.



GOWIN IMPLEMENTATION

Once the project has the hardware description language and the software to command the motor, the next step is implementing it on the FPGA board. All the files shall be exported on Gowin IDE.

The steps of this process are listed and explained below:

1) First, a project must be created, with the right settings specifications for our FPGA board. This project used a GOWIN GW2A-LV18 hosted on a Tang Primer 20k.
2) After that, we can proceed to add our files. Those naturally include all of our Verilog module for the motor, but also all of the .v files from the e203.

3) The constraints (.cst) file must be prepared, mapping which pins shall communicate which information and voltages inside the project. It is a simple yet crucial file that must be written carefully, any error will lead to faulty functionalities in the project, despite they are easily fixable.
In the file, we must specify the pins for the clock input (H11) in our case, for the reset button (C7), for the high- and low-side outputs to the H-bridge and so on.
This is an example of a few pins:

```
16  // --- Clock Input ---
17  IO_LOC "clk" H11; // 27MHz clock input from on-board oscillator
18  IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=NONE;
19  /*
20  // --- Reset Input #Button S4 on the Dock --
21  IO_LOC "rst_n" C7;
22  IO_PORT "rst_n" IO_TYPE=LVCMOS33 PULL_MODE=UP;
```

4) After having uploaded all files, to test them it is necessary to select "process" in the GOWIN interface and then proceed with "synthesize", "place&route" and, if all is free of errors, we must open the programmer window, connect the board to the computer via USB and finally upload the whole project.

Sadly, we couldn't test our project on the motor before leaving Tongji university. Our tests and simulations have been 100% virtual and our only attempt on the physical motor was unsuccessful since we were still at an initial stage of the project. We wish we could've tested it on the real motor as well and hope we will manage to if we have access to the laboratory. For this reason, the steps described on Gowin have only partially been completed on our side, and we limited to testing the whole project, connected onto the e203 hummingbird CPU, on Visual Studio Code.

CONCLUSION

This project has been by far the most complicated we had to face with many new concepts, complexities and devices to be used together to make a functioning project. Creating all the Verilog files to drive the motor together already had been a major challenge, only to discover, once we had tested them individually and together, that we were not through yet and the whole implementation on the e203 hummingbird still had to be done. This project helped us deeply understand Verilog HDL and careful programming to orchestrate an harmonic collaboration between the various files to generate the output to drive the motor. We then had to focus on navigating the numerous, kilometric files of the e203 to make the correct port connections to bring inputs and outputs through the whole CPU and connect them to the board. The C coding as well has been a major surprise and a use of C coding language in a manner we had never seen before, to directly write in registers specifically stored in precise memory addresses, taking care of the most specific details to transmit the correct bits and bytes in the correct area. This project has been very variegated also for the amounts of tools and software used to create every single part of the necessary files. We are disappointed to not have been able to drive our electric motor in the real world but are still satisfied with the final work.

REFERENCES&SOURCES

https://www.embitel.com/blog/embedded-blog/brushless-dc-motor-vs-pmsm-how-these-motors-and-motor-control-solutions-work

Embedded - The C Programmers Guide to Verilog

https://projectf.io/posts/fpga-sine-table/

1. Overview — Hummingbirdv2 E203 Core and SoC 0.2.1 documentation

Google AI Studio – ChatGPT – DeepSeek AI tools

Thanks to professor Jiang Lei for his invaluable and patient guidance throughout the project.

In the Appendix, the following files are available:

- Tang Primer 20k and BLDC H-Bridge Schematics
- Constraints file (.cst) for our project
- *Class notes* showing the project's structure

And of course, the whole project, with all .v and .c files.

Francesco Tassi, 2359266

Flavio Davide Borriello, 2359270

同济大学, July 2025, Shanghai